

# Taazur HRMS (hrms-app)

Independent full-scope holistic audit — security, RBAC & multi-tenancy, data & database, API, business logic, UI/UX, accessibility, performance, privacy and test coverage.

61/100

Significant improvement required

Overall weighted health score

|                        |                                           |
|------------------------|-------------------------------------------|
| APPLICATION            | ENVIRONMENT                               |
| Taazur HRMS (hrms-app) | Production demo (hrms-app-chi.vercel.app) |
| AUDIT DATE             | METHOD                                    |
| 2026-07-21             | White-box (source) + live black-box       |
| TOTAL FINDINGS         | CRITICAL / HIGH                           |
| 90                     | 4 / 18                                    |

This document contains confidential security findings. Distribute on a need-to-know basis. Passwords, tokens and secrets are masked. Data shown is from a fictitious demo tenant.

# 1. Executive summary



This is an independent, full-scope holistic audit of Taazur HRMS (hrms-app), a multi-tenant Saudi HR and payroll SaaS built on Next.js 16, tRPC, Drizzle and schema-per-tenant PostgreSQL. It was performed two ways at once: a white-box review of the entire source tree by eight specialist reviewers, and live black-box testing plus multi-role screenshot capture against the running production demo. It produced 99 raw findings, de-duplicated to 90 distinct issues (cross-domain corroboration is noted on each finding).

The application has a genuinely strong security core. Authorization is enforced server-side on the tRPC API — an unauthenticated API call returns 401, verified live. Tenant isolation is schema-per-tenant, resolved from the session and never from client input. Personally identifiable data (iqama, passport, IBAN) is encrypted at rest with AES-256-GCM behind a fail-closed production key. The live site ships a complete modern security-header set (HSTS preload, CSP, X-Frame-Options DENY, frame-ancestors none), disables source maps, and 404s its debug endpoint in production. A prior internal RBAC audit scored the app 90/100.

That 90/100 does not survive a full re-audit. The strong controls live almost entirely in the tRPC layer; a handful of custom REST routes bypass them. Four Critical issues result: (1) an unauthenticated endpoint that returns the entire tenant registry — every tenant's name, commercial-registration number and internal Postgres schema name, plus recent users' emails and roles (confirmed live); (2) an unauthenticated proxy to the Saudi Qiwa government API using server credentials; (3) and (4) a tenant-provisioning path that creates only 2 of roughly 24 tables and migrations hard-coded to a single development schema, leaving any newly signed-up tenant structurally broken. The current demo tenant was hand-seeded and is unaffected by (3) and (4).

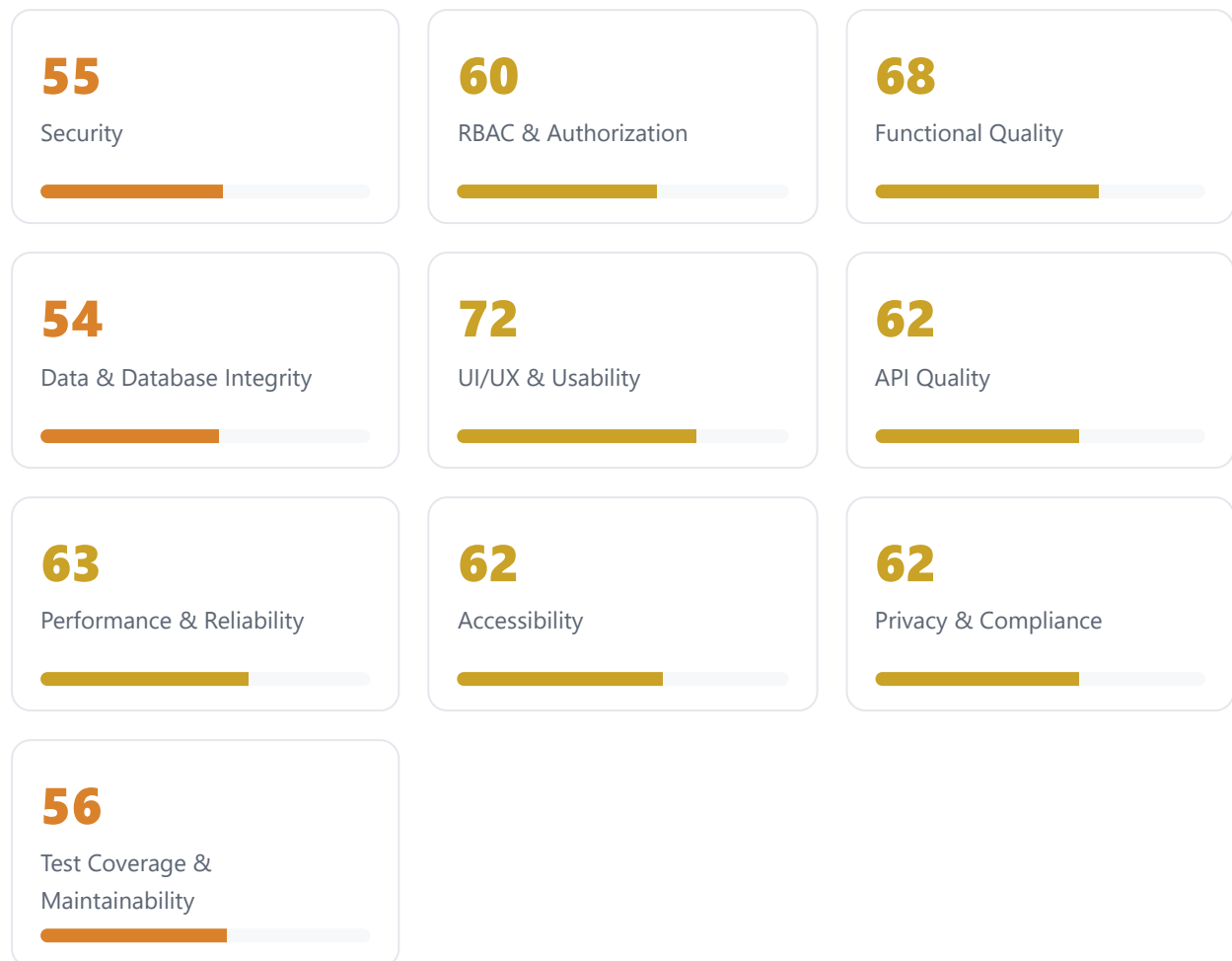
Beyond access control, the audit found business-logic gaps (expense self-approval; a leave-accrual routine that compounds balances to roughly 6.5x entitlement; duplicate end-of-service settlements), data-integrity and migration drift, incomplete audit-logging of security events, and a real production database password sitting in a gitignored file on disk. UI/UX and accessibility are above average — Radix primitives, well-built forms, thoughtful empty states — but with right-to-left layout breakage and a dead Arabic language toggle on the login page. Engineering hygiene is mixed: typecheck is clean and 152 of 159 unit tests pass, but the tenant-isolation regression tests are mostly skipped and a high-severity drizzle-orm SQL-injection advisory is unpatched.

The overall weighted health score is 61/100 — 'significant improvement required'. Crucially, this reflects real, currently-reachable exposure (chiefly the two unauthenticated endpoints), not a weak foundation. Almost every Critical and High item is a small, surgical fix: delete or gate about four REST routes, correct the provisioning DDL, add 'actor is not the subject' checks to a few approvals, and rotate one credential. None require architectural change. A focused re-test after the Immediate and Urgent items in Section 9 should return the product to the low-to-mid 80s.

**Most serious issue:** Unauthenticated GET /api/auth/check-tenants returns every tenant's name, CR number and internal Postgres schema name, plus recent users' emails and roles across all tenants — confirmed live. On a real multi-customer deployment this is a cross-tenant personal-data breach reachable by anyone with the URL.

## 2. Overall scorecard

Module scores (0–100), weighted into the overall health score shown on the cover.



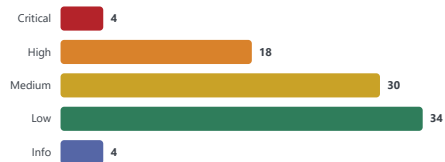
### Test coverage

| Metric                         | Value                                                       |
|--------------------------------|-------------------------------------------------------------|
| Pages / routes discovered      | 80+ (auth/public + 60+ dashboard)                           |
| Roles exercised live           | 4 of ~8 (HR Manager, HR Specialist, Dept Manager, Employee) |
| tRPC routers reviewed          | 22 (all)                                                    |
| REST route handlers reviewed   | 22 (all)                                                    |
| Live screenshots captured      | 45 across 4 roles / 4 viewports                             |
| Findings (raw / de-duplicated) | 99 / 90                                                     |
| Unit tests                     | 152 passed, 7 skipped, 1 suite env-failure (152/159)        |
| Typecheck / Lint               | PASS (17 pkgs) / 372 lint errors (web only)                 |
| Dependency audit               | 1 high (drizzle-orm), 4 moderate                            |

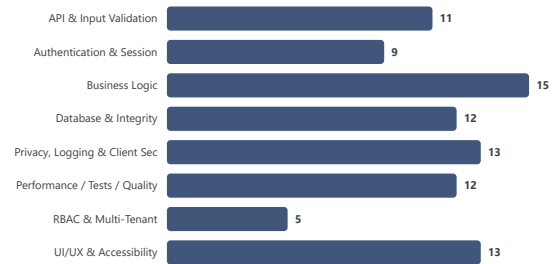
### 3. Severity, category & domain distribution

---

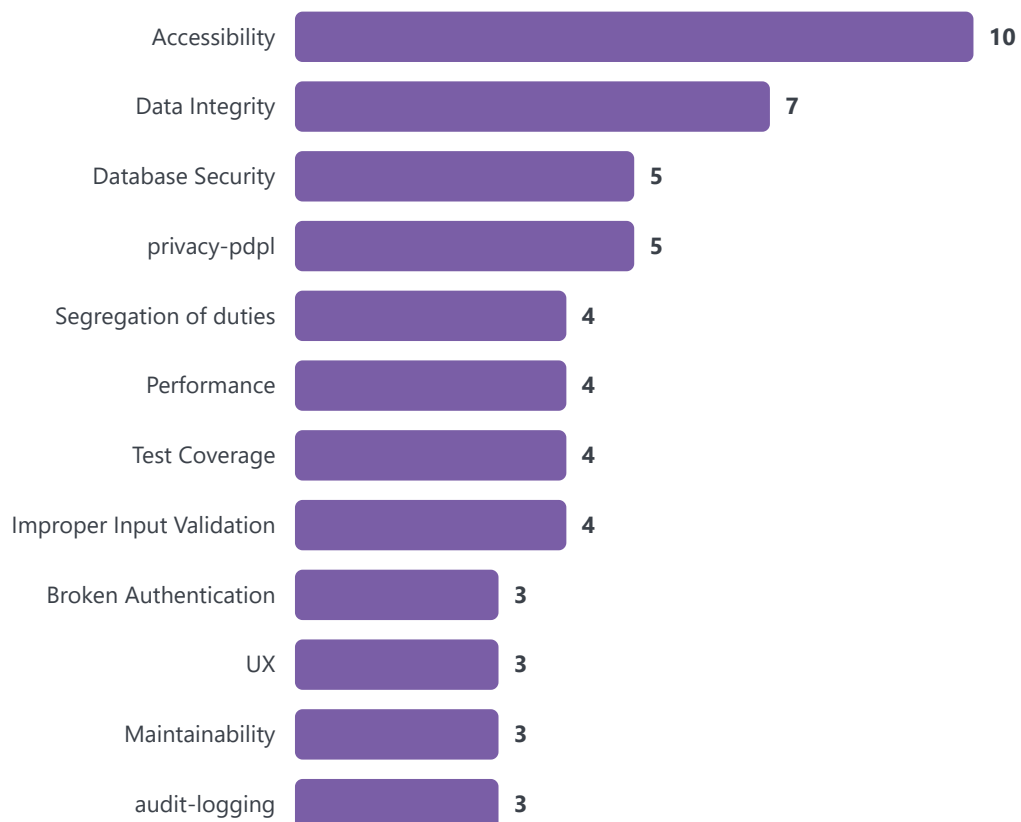
#### By severity



#### By audit domain

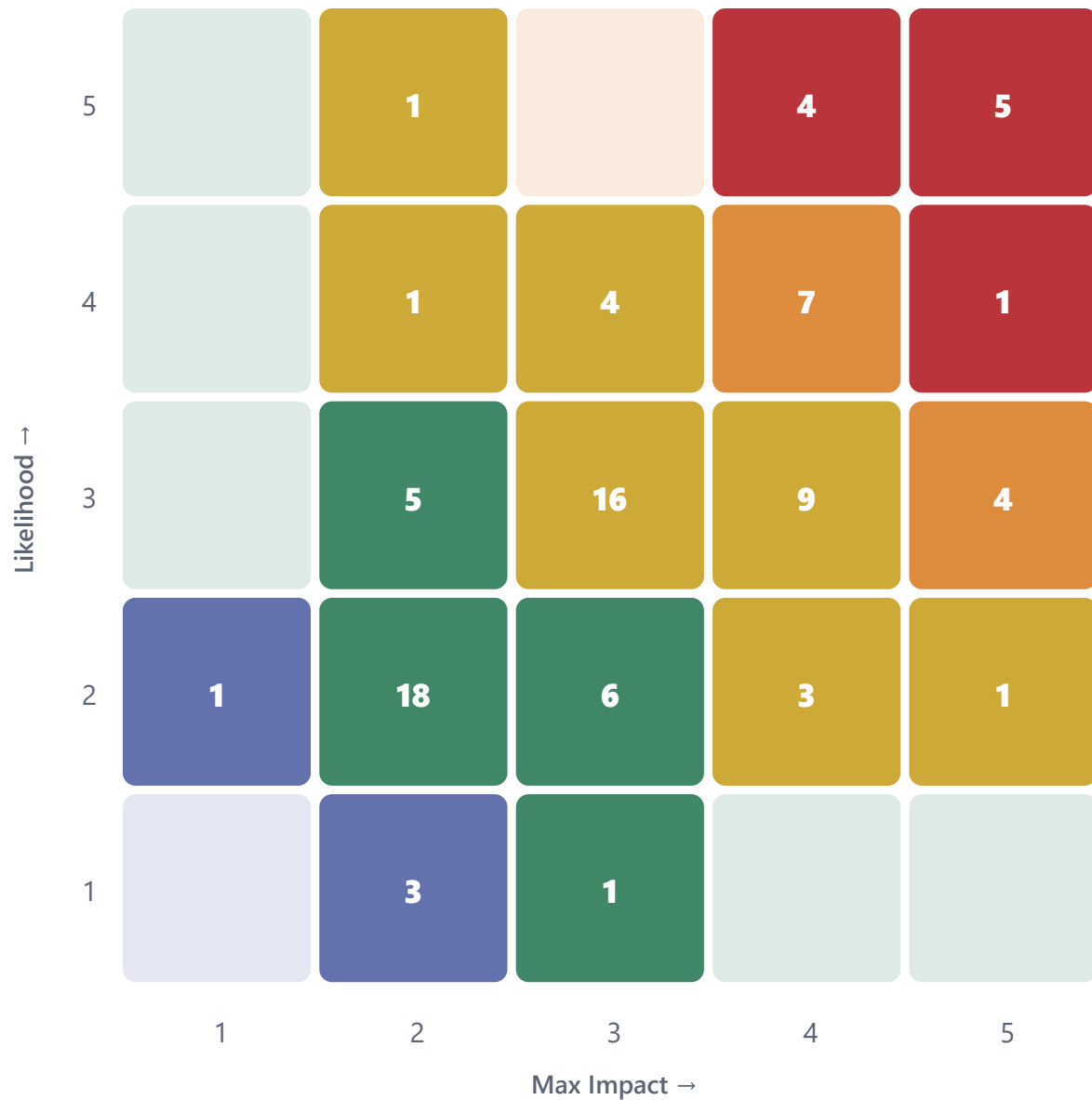


#### By category (top)



## 4. Risk heat map

Each cell counts findings at that likelihood × maximum-impact. Colour follows Likelihood × Impact (20–25 Critical, 15–19 High, 8–14 Medium, 3–7 Low).



## 5. Scope, methodology & limitations

---

Target: the Taazur HRMS (hrms-app) monorepo and its production demo at hrms-app-chi.vercel.app. The demo runs a fictitious 'Rukn Energy Services' tenant with four public demo logins; the audit was authorized by the product owner.

This is a full independent re-audit across all 18 audit phases — not limited to, and not trusting, the prior internal RBAC audit. Two surfaces were tested: white-box (the complete source tree) and live black-box (HTTP probing plus authenticated Playwright screenshot capture on the running demo).

Roles exercised live: HR Manager, HR Specialist, Department Manager and Employee — all four demo accounts logged in successfully and were walked through their accessible pages.

### Methodology

- Eight parallel specialist reviewers: authentication & session; RBAC & multi-tenant isolation; API & input validation; database & data integrity; business logic; UI/UX & accessibility; privacy, logging & client security; and performance, tests & code quality. Every finding is grounded in file:line source evidence.
- Independent re-verification of the prior RBAC audit's 13 fixes — each was checked against live code rather than trusted.
- Live black-box probing: security headers, TLS/HSTS, source-map exposure, unauthenticated endpoint enumeration, and error/404 states (non-destructive GETs only).
- Live authenticated capture: 45 screenshots across 4 roles and 4 viewports, including RBAC-denial evidence for an employee attempting restricted routes.
- Severity assigned on the P0–P4 matrix; Risk score = Likelihood x max(Technical, Business, User impact). Cross-domain duplicate findings were merged (9 suppressed from counts, corroboration recorded).
- Static build, unit-test, lint and dependency-audit commands were executed; results are in Section 2.

### Limitations & coverage caveats

- No direct database access was used; all database findings are reasoned from schema and migration source and are marked where they could not be verified against a live database.
- Live testing used the demo tenant with fictitious data only. No destructive tests were run and no records were created on the live application. Authenticated privilege-escalation (e.g. the invite route) and the /api/qiwa impact were verified in code, not exploited; the Qiwa live impact also depends on government credentials being configured.
- The local unit-test run had one suite fail solely because of a missing local DATABASE\_URL (a known-broken local environment), not a code defect.
- The untracked root .env and apps/web/.env could not be read (permissions); related items are marked unable-to-verify.
- Accessibility was assessed from markup, not a full assistive-technology pass; colour contrast was not measured at runtime.

- Only 4 of roughly 8 roles have public demo logins, so live role-by-role coverage is partial; RBAC for the remaining roles was assessed from source.

## 6. RBAC permission matrix

Roles: a platform-operator gate (PLATFORM\_ADMIN\_EMAILS allowlist), plus the per-tenant roles super\_admin, hr\_manager, hr\_specialist, payroll\_admin, department\_manager, recruiter, employee and candidate. The capability model in packages/auth/src/rbac.ts is the intended source of truth.

The tRPC layer enforces this model well. The failures below concentrate in custom REST routes that bypass tRPC entirely, and in a few procedures that use companyProcedure (which only blocks employee/candidate) instead of an explicit capability check. Rows marked PASS were verified as correctly enforced, including some the prior audit fixed.

| Module · Action                            | Expected access                        | Actual access (lowest-privilege with access)                     | Result             |
|--------------------------------------------|----------------------------------------|------------------------------------------------------------------|--------------------|
| /api/auth/check-tenants · list all tenants | Platform-operator only                 | Any anonymous visitor — all tenants + schema names + user emails | FAIL — RBAC-001    |
| /api/qiwa · gov sync / create / terminate  | HR / payroll, tenant-scoped            | Any anonymous visitor, using server credentials                  | FAIL — API-001     |
| /api/auth/login · password check           | Throttled + lockout + MFA              | Unthrottled, no lockout, no MFA (password oracle)                | FAIL — AUTH-002    |
| employee.getByld · read salary + PII       | HR / payroll; dept-scoped for managers | Recruiter & Dept Manager read any employee's salary/PII          | FAIL — API-004     |
| expense.approve · approve a claim          | A different approver (not submitter)   | Submitter can approve their own claim                            | FAIL — BIZ-001     |
| expense.markPaid · mark paid               | HR / payroll only                      | Any staff role (recruiter, department manager)                   | FAIL — BIZ-002     |
| attendance / leave · list & reports        | Capability-gated (HR / manager scope)  | Any staff role incl. recruiter (companyProcedure)                | FAIL — RBAC-004    |
| payroll · view runs / payslips             | PAYROLL_VIEW_ROLES only                | PAYROLL_VIEW_ROLES only                                          | PASS               |
| /super-admin · cross-tenant registry       | Platform-operator (allowlist)          | Platform-operator (allowlist)                                    | PASS               |
| employee.create / update · write           | requireRole (HR)                       | requireRole (HR); field-scoped update                            | PASS               |
| /api/company/invite · invite with role     | Role-gated, cannot mint super_admin    | Role-gated allowlist (SEC-001 verified fixed)                    | PASS               |
| tRPC · any query while unauthenticated     | 401 Unauthorized                       | 401 Unauthorized (verified live)                                 | PASS               |
| MFA · privileged roles                     | Enforced for admins                    | Opt-in only; never enforced by policy                            | PARTIAL — AUTH-006 |

| Module · Action                           | Expected access         | Actual access (lowest-privilege with access) | Result             |
|-------------------------------------------|-------------------------|----------------------------------------------|--------------------|
| Session · after role change / offboarding | Revoked on next request | Stale up to 30 min; no revocation mechanism  | PARTIAL — AUTH-003 |

## 7. Findings by domain

---

### Authentication & Session — score 62/100

Solid primitives — bcrypt cost 12, hashed single-use 1-hour reset tokens, no user enumeration, durable lockout with backoff, correct TOTP. Undermined by a second unthrottled login endpoint, an unauthenticated tenant/user disclosure route, reset tokens written to logs, no absolute session cap or role-revocation, and MFA that is opt-in only.

### RBAC & Multi-Tenant Isolation — score 60/100

The tRPC authorization core and schema-per-tenant isolation are genuinely strong and most of the prior audit's fixes hold. But the re-audit re-opened SEC-002 and SEC-009 through unauthenticated 'twin' REST routes the earlier audit never enumerated, and found salary/PII over-exposure and un-scoped manager reads. The 90/100 does not hold once the REST surface is weighted in.

### API & Input Validation — score 62/100

A well-built tRPC core (~95% Zod-validated, no SQL-injection sinks, object-level authz, CSV formula-injection neutralized) with a few unauthenticated REST side-doors (qiwa, check-tenants, login) and an upload path-traversal via an unsanitized category field.

### Database & Data Integrity — score 54/100

Cryptographically sound PII codec, money columns all numeric, and a well-built append-only audit trigger. But three schema mechanisms disagree: runtime tenant provisioning creates only 2 of ~24 tables, two migrations hard-code a single dev schema, and instant columns are naive timestamp (not timestamptz) — so new tenants are broken and time data is timezone-ambiguous.

### Business Logic — score 68/100

Server-side payroll/GOSI/end-of-service computation, a period lock with audited reopen, and statutory deduction caps are commendable. Weak points: 'actor is not the subject' is not enforced (expense/leave/comp self-approval), missing idempotency/uniqueness guards (duplicate settlements), a compounding leave-accrual bug, and audit coverage present only on payroll/settlement.

### UI/UX & Accessibility — score 72/100

Above-average craft — accessible Radix primitives, an exemplary login form, thoughtful empty states, focus-visible defaults. Gaps: the dashboard shell hardcodes physical offsets so RTL/Arabic layout breaks, the login-page Arabic toggle is a dead button, several forms miss label associations, and two destructive deletes fire with no confirmation. Usability 74 / Accessibility 62.

### Privacy, Logging & Client Security — score 62/100

Strong posture — AES-256-GCM PII at rest, complete security headers, no client-side token/PII storage, a role-gated tenant-scoped audit API. Weaknesses: CSP allows unsafe-inline scripts, the

audit trail omits login/role/export/PII-view events, a salary certificate decrypts iqama with no audit entry, no PDPL erasure/rectification/consent flow, and a real prod DB password is present in a gitignored on-disk file.

## **Performance, Tests & Quality — score 60/100**

Clean typecheck across all packages, consistent pagination, a deep payroll test suite. But the employees table has zero indexes, attendance.getSubtree fetches the whole tenant per call, an N+1 sits in leave accrual, external calls lack timeouts/retries, tenant-isolation tests are mostly skipped, the two largest routers are untested, lint fails with 372 errors, and drizzle-orm is below its SQL-injection patch.

## 8. Detailed findings

90 findings, ordered by severity then risk score. Evidence cites `file:line` from the audited source.

**API-001**

**Critical · P0**

**Risk 25**

**confirmed**

### Unauthenticated proxy to Qiwa government API (/api/qiwa)

**corroborated: RBAC-002**

**Module:** REST API - Qiwa integration · **Role:** anonymous · **L5 ·T4 B5 U4**

The REST route `apps/web/app/api/qiwa/route.ts` exposes GET, POST and PUT handlers that acquire a Qiwa OAuth token and proxy the request straight to the Qiwa government API (`api.qiwa.sa / sandbox`). No `auth()` session check, no role check, no tenant scoping and no Zod validation are performed. The path is also NOT included in `middleware.ts config.matcher`, so the global rate-limit and same-origin CSRF checks never run for it. The request body is forwarded verbatim to the Qiwa endpoint (`JSON.stringify(body)`) and the employee id comes from an unvalidated `?id=` query param. A properly gated tRPC equivalent (`qiwaRouter, requireRole(...QIWA_MANAGE_ROLES)`) already exists, so this is an unprotected parallel door to the same integration.

**Expected:** Every call to a government-integration endpoint must require an authenticated session, enforce the qiwa manage roles, be tenant-scoped, validate the payload with the existing `qiwaEmployeeSchema`, and be covered by rate limiting.

**Actual:** Any unauthenticated caller can POST/PUT arbitrary employee data to Qiwa or GET employee records, using the server's own Qiwa credentials, with no session, role, tenant, validation or rate limit.

#### ► Evidence (2)

**Recommendation:** Add `auth()` + role check (`QIWA_MANAGE_ROLES`) at the top of every handler, resolve tenant from the session, validate the body with `qiwaEmployeeSchema/qiwaContractSchema`, add `/api/qiwa` to `middleware.matcher` for rate-limit + CSRF, and return generic error messages. Preferably retire this REST route in favour of the already-gated tRPC `qiwaRouter`.

**Workaround:** Block `/api/qiwa` at the edge/WAF or delete the route until authentication is added.

**M**

**immediate**

**Backend/API team**

**DB-001****Critical · P0**

Risk 25

confirmed

## Tenant provisioning DDL creates only 2 of ~24 module tables — new tenants are structurally broken

**Module:** packages/db tenant-manager · **Role:** system / new tenant admin · L5 · T5 B5 U5

generateTenantDDL() emits CREATE TABLE for only `departments` and `employees`. The tenant schema module defines ~24 tables (payroll\_runs, payslips, wage\_files, expenses, leave\_types/requests/balances, attendance shifts/records/exceptions, documents, compliance\_checks, final\_settlements, notifications, audit\_logs, recruitment, retention, ai, qiwa\_contracts, policy\_documents, guide\_maps, employee\_invitations, employment\_history). None of the other tables are created when a tenant is provisioned at runtime.

**Expected:** createTenantSchema() should materialize the complete, current tenant table set (every table exported from schema/tenant/index.ts) with all constraints, indexes and triggers, so all modules work for a freshly signed-up tenant.

**Actual:** Only departments + employees are created. Payroll, leave, attendance, expenses, recruitment, audit, compliance etc. tables do not exist for a new tenant, so those modules fail with 'relation does not exist' until someone hand-runs a seed/DDL script.

### ► Evidence (3)

**Recommendation:** Generate tenant DDL from the Drizzle schema (single source of truth) rather than a hand-maintained string. Options: (a) drive drizzle-kit against each new tenant schema via SET search\_path; (b) build a per-tenant migration runner that replays an authoritative ordered DDL bundle covering all tables + constraints + indexes + triggers; (c) at minimum, extend generateTenantDDL to the full table set and add a test asserting parity with schema/tenant/index.ts exports.

**Workaround:** Run scripts/seed-rukn-modules.ts (PHASE 1 DDL) against each new tenant manually.

L (2-4 days: author authoritative per-tenant DDL/migration mechanism + parity test)

Immediate

Backend / DB platform

**DB-002****Critical · P0**

Risk 25

confirmed

**Migrations 0001 & 0002 hardcode a single dev tenant schema (tenant\_1ed8b6bd3743) — changes never reach other or new tenants****Module:** packages/db drizzle migrations · **Role:** system · L5 · T5 B4 U5

Migration 0001 (immigration + gcc columns) and 0002 (policy\_documents table + indexes) are written against a literal schema name `tenant\_1ed8b6bd3743`. They ALTER/CREATE only inside that one developer tenant. Any other existing tenant and every future tenant never receives these columns/tables. Contrast with 0006/0007/0009 which correctly loop over information\_schema.schemata LIKE 'tenant\\_%`.

**Expected:** Schema migrations must apply to every tenant schema (loop over all tenant\_<hex> schemas) and be baked into new-tenant provisioning.

**Actual:** 0001/0002 mutate exactly one schema. Employees.immigration\_status / visa\_type / iqama\_expiry / passport\_expiry / occupation\_code / skill\_level / gcc\_status and the policy\_documents table exist only in tenant\_1ed8b6bd3743.

**► Evidence (2)**

**Recommendation:** Rewrite 0001/0002 as DO-loops over all tenant\_<hex> schemas (as 0006/0007/0009 do) AND fold the columns/tables into the authoritative provisioning DDL from DB-001. Add a CI check that fails on any literal tenant\_<hex> reference in migrations. Also fix the date-vs-text mismatch (see DB-005).

**Workaround:** Manually replay the ALTERs against every existing tenant schema. **M (1-2 days)**

Immediate

Backend / DB platform

**RBAC-001****Critical · P0**

Risk 25

confirmed

**Unauthenticated cross-tenant registry disclosure via GET /api/auth/check-tenants (re-opens SEC-002, leaks schemaName)**

corroborated: AUTH-001, API-002, live-probe

**Module:** REST route (custom) · **Role:** anonymous (no session required) · L5 · T5 B5 U4

The custom route GET /api/auth/check-tenants returns the ENTIRE tenant registry (companyName, crNumber, nitaqatActivity, planTier, regulatoryContext AND internal Postgres schemaName) plus the 10 most recent users across ALL tenants (email, name, role, tenantId) with NO authentication, no role check and no tenant scoping. This is the exact data SEC-002 was remediated for on the tRPC procedure auth.tenantsList (now gated to PLATFORM\_ADMIN\_EMAILS with schemaName removed) — but this parallel REST door bypasses that fix entirely and additionally re-exposes schemaName, the internal isolation key SEC-002 explicitly removed. The path is not in the middleware matcher, so no rate-limit or edge guard applies either.

**Expected:** Cross-tenant registry data must be restricted to a true platform operator (same PLATFORM\_ADMIN\_EMAILS boundary used by auth.tenantsList) and must never expose schemaName. Ideally this diagnostic route should be deleted.

**Actual:** Any anonymous HTTP client receives every tenant's company name, CR number, plan tier and internal schemaName plus recent user emails/roles/tenantIds.

## ► Evidence (3)

**Recommendation:** Delete apps/web/app/api/auth/check-tenants/route.ts. If a diagnostic is required, gate it on the same PLATFORM\_ADMIN\_EMAILS operator boundary as auth.tenantsList and never return schemaName. Add the path to the middleware matcher regardless.

**Workaround:** Block /api/auth/check-tenants at the edge/CDN until removed.

S (delete route)

Immediate (hotfix)

Backend / Platform

**BIZ-001**

High · P1

Risk 20

confirmed

**Expense self-approval: submitter can set themselves as the approver and approve their own claim**

**Module:** expense · **Role:** department\_manager, hr\_specialist, payroll\_admin, recruiter (any non-employee staff role) · L4 · T3 B5 U2

createExpenseSchema accepts a client-supplied approverEmployeeId with no check that it differs from the submitter. The approve mutation authorises the row's approverEmployeeId (or any HR role). A submitter who sets approverEmployeeId to their own employee id therefore becomes the approver of their own expense and can approve it. Plain 'employee' role is blocked from expense.approve by canAccessProcedure, but every staff role above employee (department\_manager, hr\_specialist, payroll\_admin, recruiter) reaches approve via protectedProcedure.

**Expected:** An expense approver must be a different person from the submitter (actor != target). A self-referential approver should be rejected at submission, and approve must refuse when approverEmployeeId === the row's employeeId.

**Actual:** No actor != target guard exists. approverEmployeeId is taken verbatim from input; approve only checks that the caller is the assigned approver or HR, so a self-assigned approver passes.

**► Evidence (3)**

**Recommendation:** Reject approverEmployeeId === submitter at create; in approve, additionally block when existing.employeeId === existing.approverEmployeeId or when the acting HR user is the submitter. Enforce a two-person rule for any monetary approval.

**Workaround:** Restrict approve/markPaid to a dedicated finance role and monitor the audit log for same-actor submit+approve pairs (not currently logged — see BIZ-003).

S (0.5-1 day)

next

Backend / Payroll squad

**DB-003****High · P1****Risk 20****confirmed**

## **audit\_logs table and its append-only trigger are not part of tenant provisioning — new tenants have no audit trail and no tamper protection**

**Module:** packages/db tenant-manager + audit\_logs · **Role:** system / compliance · **L5 · T4 B5 U3**

generateTenantDDL does not create audit\_logs, so new tenants have no audit table at all. Migration 0007 installs the append-only BEFORE UPDATE/DELETE trigger only for schemas that ALREADY contain audit\_logs, and runs once at migration time. A tenant created after 0007 was applied therefore gets neither the table nor the trigger. The trigger design itself is correct and idempotent, but it is not wired into provisioning.

**Expected:** Every tenant should be created with audit\_logs plus the prevent\_audit\_mutation() trigger so audit entries are immutable from day one.

**Actual:** New tenants: no audit\_logs table (audit writes fail or are skipped). Even if the table were added later by a script, the append-only trigger is not re-installed for tenants created after 0007.

### ► Evidence (3)

**Recommendation:** Add audit\_logs (with actor/action/target/before/after/ip/timestamp) and the prevent\_audit\_mutation() trigger to the authoritative provisioning DDL (DB-001). Keep 0007 as the backfill for legacy schemas. Add an integration test asserting UPDATE/DELETE on audit\_logs raises in a freshly provisioned tenant.

**Workaround:** Run the 0007 DO-block after every tenant creation.

**S (0.5-1 day)****Immediate****Backend / Security**

**DB-005****High · P1****Risk 20****confirmed****Provisioned employees table is missing immigration/gcc columns the app requires, and migration types (text) diverge from schema (date)****Module:** packages/db employees · **Role:** system / hr\_manager · **L5 · T4 B4 U4**

The Drizzle employees schema defines immigration columns (passportExpiry, iqamaExpiry, exitReentryExpiry as `date`; visaType, occupationCode, skillLevel, immigrationStatus, gccStatus). generateTenantDDL omits all of these. Migration 0001 adds them ONLY to tenant\_1ed8b6bd3743 (see DB-002) AND declares passport\_expiry/iqama\_expiry/exit\_reentry\_expiry as `text`, conflicting with the `date` type in the canonical schema. So new tenants lack the columns; the one hardcoded tenant has them with the wrong type.

**Expected:** Provisioned employees table matches employees.ts exactly, including immigration columns with correct `date` types.

**Actual:** New tenants: columns absent.  
Hardcoded tenant: columns present but typed text where Drizzle expects date -> type coercion / comparison bugs for expiry logic.

**► Evidence (2)**

**Recommendation:** Include immigration/gcc columns with correct date/enum types in the authoritative provisioning DDL; align 0001 to date types; backfill/convert text->date on the affected schema.

**Workaround:** Manually add columns (as date) to affected tenant schemas.

M (1-2 days incl. type conversion)

Immediate

Backend / DB platform

**UX-001****High · P1****Risk 20****confirmed**

## Dashboard shell uses hardcoded physical left/padding-left, so layout does not mirror in RTL (Arabic)

**Module:** Dashboard Shell / RTL · **Role:** All authenticated roles · L5 · T3 B4 U4

globals.css states the intent explicitly: "RTL: every component is RTL-first; dir=\"rtl\" on <html> flips physical properties ... so layouts mirror automatically." DashboardShell does correctly toggle document.documentElement.dir/lang at runtime (dashboard-shell.tsx:35-36), but the shell's own layout uses hardcoded physical Tailwind classes instead of logical ones: the sidebar wrapper is pinned with `left-0` and the content column uses `md:pl-[278px]`. Neither responds to `dir="rtl"`.

**Expected:** When preferredLanguage is switched to Arabic (dir=rtl), the sidebar should move to the visual right edge and the main content column should reserve space on the right (e.g. via `inset-inline-start-0` / `ps-[278px]` logical utilities, or `[dir=rtl]:` overrides), matching the RTL-first design intent documented in globals.css.

**Actual:** The sidebar stays pinned to the physical left of the viewport and main content keeps its left padding even after switching to Arabic, producing a left-docked nav in an otherwise right-to-left reading page.

### ► Evidence (3)

**Recommendation:** Replace `left-0` / `pl-[278px]` (and similar physical positioning in header.tsx, employees list search icon, etc.) with Tailwind logical-property utilities (`start-0`, `ps-[278px]`) or add `[dir=rtl]:right-0 [dir=rtl]:pr-[278px]` overrides, then visually QA every dashboard page in Arabic.

**Workaround:** None for end users; Arabic users can still complete tasks, just with a mirrored-incorrectly nav rail.

[M](#)[Next](#)[Frontend / Design system](#)

**UX-002****High · P1****Risk 20****confirmed**

### Language toggle on the login page is non-functional (dead button)

**Module:** Auth · **Role:** Unauthenticated / all · **L5 · T2 B4 U4**

The login page renders what looks like a working EN/Arabic toggle in the top bar, but the "العربية" element is a `` with no `onClick` handler at all, and "EN" is a plain ``, not an interactive control. Every other authenticated page in the app has a working language toggle (header.tsx uses `setPreferredLanguage`), so this looks like an oversight specific to the pre-login screen.

**Expected:** Clicking "العربية" switches the login page (and ideally the whole pre-auth flow) to Arabic/RTL, consistent with the working toggle used everywhere post-login.

**Actual:** Clicking the "العربية" button does nothing; there is no state, no dir change, and no visual feedback of any kind.

#### ► Evidence (2)

**Recommendation:** Wire the login-page toggle to the same `Lang`/dir-switching mechanism used in DashboardShell, or at minimum route users to a `?lang=ar` variant of the login form with translated copy.

**Workaround:** Users can complete login in English and switch language after signing in.

**S****Next****Frontend**

**API-004**

High · P1

Risk 16

confirmed

**employee.getById / employee.list over-select salary + PII and lack object-level scoping**

corroborated: RBAC-003

**Module:** tRPC - employee router · **Role:** department\_manager, recruiter · L4 · T3 B4 U4

employee.getById and employee.list run on companyProcedure, which only excludes the employee and candidate roles - recruiter and department\_manager are admitted. Both return the whole employees row (no column projection), including plaintext salaryBasic/salaryHousing/salaryTransport and the \*\_enc PII columns (iqamaNumberEnc, passportNumberEnc, bankIbanEnc). Unlike employee.list (which department-scopes a department\_manager), employee.getById performs NO department scoping, so a department\_manager or recruiter can fetch ANY employee's full record - including salary - by iterating UUIDs. This overlaps RBAC-004 (recruiter/department\_manager must not see salary) but is reported here as a data-layer over-selection + missing object-level authorization issue.

**Expected:** Reads should project only the columns the caller is entitled to (salary and \*\_enc PII stripped for recruiter/department\_manager) and getById should enforce the same department scoping as list.

**Actual:** getById returns the entire employee row (salary + encrypted PII + payslips relation) to any company role with no department scoping; a recruiter sees salary they are explicitly not permitted to see.

## ► Evidence (2)

**Recommendation:** Add a column allowlist (Drizzle columns projection) that strips salary and \*\_enc fields for non-payroll roles, and apply getManagedDepartmentIds scoping to employee.getById as is already done in employee.list. Coordinate with the RBAC owner (RBAC-004).

**Workaround:** Restrict getById/list to the salary-authorized roles until per-column projection is added.

M

next

Backend/API team + RBAC owner

**AUTH-002**

High · P1

Risk 16

confirmed

**Orphan POST /api/auth/login is an unthrottled password-verification oracle (no rate limit, no logout, no MFA)**

corroborated: RBAC-006, API-003

**Module:** apps/web/app/api/auth/login · **Role:** anonymous · L4 · T4 B4 U4

A second, custom credentials endpoint exists alongside the NextAuth flow. It validates the password with bcrypt but is NOT in the middleware matcher (so the 5/min login rate limit never applies), does NOT consult or increment the durable account-logout state, and does NOT enforce MFA. It therefore permits unlimited online password guessing against real accounts and, on success, discloses the account role and tenantId. The frontend uses signIn()/callback/credentials instead, so this route is unreferenced dead code that still serves live traffic.

**Expected:** All credential-checking endpoints must share the same brute-force controls (IP rate limit + durable per-account logout + MFA gate) as the NextAuth authorize() path, or not exist.

**Actual:** The route calls bcrypt compare and returns 200/401 with no throttling, no logout accounting, and no MFA. It is missing from middleware.ts matcher entirely.

**► Evidence (3)**

**Recommendation:** Delete this route (the app authenticates via NextAuth). If a JSON login API is genuinely needed, route it through the same logout + rate-limit + MFA logic and add it to the middleware matcher.

**Workaround:** Add /api/auth/login to the middleware matcher with the login rate limit, or block it at the edge. S (1-2h) immediate Backend/Auth

## Monthly leave accrual compounds: year-to-date days are added to the existing balance on every run

**Module:** leave (accrual engine) · **Role:** system (cron) / hr\_manager · L4 · T3 B4 U2

calculateAccrual computes  $\text{daysAccrued} = (\text{daysAllowed}/12) * \text{monthsElapsed}$ , where  $\text{monthsElapsed} = \min(12, \text{currentMonth}+1)$  is the YEAR-TO-DATE month count, then sets  $\text{newBalance} = \text{previousBalance} + \text{daysAccrued}$ . Because the YTD figure is ADDED to the already-accrued balance every time the job runs, running the monthly cron each month compounds the balance to  $\text{daysAllowed} * (1+2+\dots+12)/12 = \text{daysAllowed} * 6.5$  by December (e.g. 21 days  $\rightarrow$  ~136 days). Correct only if run exactly once per year.

**Expected:** A monthly accrual run should add one month's entitlement ( $\text{daysAllowed}/12$ ) to the balance, and be idempotent per month (no double-credit if re-run).

**Actual:** It adds the YTD accrual to the prior balance, so periodic runs over-accrue geometrically.

### ► Evidence (3)

**Recommendation:** Make the monthly path grant exactly one month ( $\text{daysAccrued} = \text{monthlyAccrual}$ ) and add idempotency keyed to (employee, leaveType, year, month) so a re-run in the same month is a no-op; add unit tests asserting  $\text{balance} == \text{entitlement}$  after 12 monthly runs.

**Workaround:** Run the accrual job at most once per year until fixed; verify balances manually.

M (1-2 days incl. tests)

next

Backend / Leave squad

**DB-004****High · P1****Risk 16****confirmed****iqama unique index and performance indexes are one-time backfills, not part of provisioning — new tenants can store duplicate national IDs and run unindexed****Module:** packages/db migrations 0006/0009 + tenant-manager · **Role:** system / hr\_manager · L4 · T4 B4 U3

The partial unique index employees\_iqama\_unique (0009) and all hot-path indexes (0006) are applied by one-time DO-loops over existing schemas. generateTenantDDL creates employees.iqama\_number\_enc as plain TEXT with no unique index and none of the perf indexes. New tenants therefore lack DB-level duplicate-iqama protection and lack indexes on notifications, employees, leave\_\*, documents, payroll, payslips, expenses, final\_settlements, compliance\_checks, departments, attendance.

**Expected:** Provisioning should create the iqama partial unique index and the standard performance indexes for every tenant.

**Actual:** Only tenants existing when 0006/0009 ran have them; new tenants have neither.

**► Evidence (3)**

**Recommendation:** Fold all indexes (incl. the iqama partial unique index) into the authoritative provisioning DDL (DB-001). Keep 0006/0009 as legacy backfills.

**Workaround:** Replay 0006 and 0009 DO-blocks after each tenant creation. **S (0.5-1 day)**

Immediate

Backend / DB platform

**QA-001****High · P1****Risk 16****confirmed**

## employees table has zero explicit indexes despite being filtered by departmentId/employmentStatus/managerEmployeeId on nearly every query

**Module:** Database schema / Employees · **Role:** all · L4 · T4 B3 U3

packages/db/src/schema/tenant/employees.ts defines the employees table with no index()/uniqueIndex() calls at all. This table is the most heavily filtered/joined table in the app: employee.ts filters on departmentId and employmentStatus and sorts by fullName/hireDate/createdAt (apps/web/trpc/routers/employee.ts:18-37); attendance.ts filters employees by departmentId (line 459) and joins via managerEmployeeId for org-tree walks; leave.ts filters active employees by employmentStatus (line 337); departments.ts joins on headEmployeeId. Postgres does not auto-index foreign-key columns (unlike the primary key), so every one of these WHERE/JOIN clauses does a sequential scan once tenant employee counts grow past a few hundred rows. Only 7 of 33 schema files (packages/db/src/schema/\*\*) define any explicit index at all; employees.ts and leave\_requests.ts, expenses.ts, documents.ts, payroll\_runs.ts, payslips.ts, notifications.ts are all in the no-index list.

**Expected:** Foreign-key / frequently-filtered columns (departmentId, employmentStatus, managerEmployeeId) should have explicit btree indexes so list/report queries scale with tenant size.

**Actual:** No index definitions exist on the employees table or on most other tenant tables that are filtered by employeeId/departmentId in routers.

### ► Evidence (3)

**Recommendation:** Add btree indexes on employees.departmentId, employees.employmentStatus, employees.managerEmployeeId, and the employeeId foreign key on leave\_requests/expenses/documents/payslips/notifications; regenerate a Drizzle migration (pnpm db:generate) rather than hand-editing SQL.

**Workaround:** None at scale currently deployed (demo tenant is small so this is not yet visible in production).

[S \(1 migration + review\)](#)[Next](#)[database-optimizer / backend](#)

**QA-008****High · P1****Risk 16****confirmed**

## Tenant-isolation regression tests are mostly skipped, not executing

**Module:** Multi-tenancy / DB isolation · **Role:** all · L3 · T4 B4 U3

The vitest run shows packages/db/src/\_\_tests\_\_/tenant-isolation.test.ts as "4 tests | 3 skipped" and packages/db/src/\_\_tests\_\_/tenant-manager.test.ts as "5 tests | 4 skipped" — i.e. the majority of the tests specifically named for verifying cross-tenant data isolation are not actually executing in the current run (likely gated behind a live-DB precondition, consistent with the known-broken local DATABASE\_URL noted in project memory).

**Expected:** Tenant-isolation tests are the most important regression guard in a multi-tenant SaaS (a prior RBAC audit already scored this area 90/100 after remediation) and should run on every CI build, not be silently skipped.

**Actual:** 7 of 9 combined tests across the two tenant-isolation-related suites are skipped in this run.

### ► Evidence (2)

**Recommendation:** Confirm whether CI provisions a real DATABASE\_URL so these suites run unskipped there; if the skip is environment-conditional, add a CI-only Postgres service (e.g. via GitHub Actions services:) so tenant isolation is verified on every PR, not just locally when a DB happens to be available.

**Workaround:** Tests exist and presumably pass when a DB is available; this is a coverage-execution gap, not a missing-test gap.

[S \(CI config\)](#)[Next](#)[qa-expert / devops-engineer](#)

**BIZ-002****High · P1****Risk 15****confirmed**

## Expense markPaid is authorised by companyProcedure, letting non-finance roles pay out expenses

**corroborated: RBAC-005****Module:** expense · **Role:** recruiter, department\_manager, hr\_specialist, payroll\_admin · **L3 · T3 B5 U1**

The markPaid mutation is documented as 'Only HR / super\_admin can do this' but is guarded by companyProcedure, which only excludes the 'employee' and 'candidate' roles. Every other role, including recruiter (who has no finance function), can mark any approved expense as paid. There is no expenses:pay capability check. Combined with BIZ-001, a single department\_manager can submit, approve, and pay one expense end-to-end.

**Expected:** Only a payroll/finance capability holder (e.g. payroll\_admin, hr\_manager, super\_admin) should close out an expense as paid, and the payer should differ from the approver/submitter.

**Actual:** companyProcedure permits department\_manager, hr\_specialist, payroll\_admin and recruiter; the code comment does not match the guard.

### ► Evidence (2)

**Recommendation:** Replace companyProcedure with requireCapability of a new 'expenses:pay' (grant only to payroll\_admin/hr\_manager/super\_admin), and add a payer != approver != submitter check. Write an audit record on payout.

**Workaround:** Temporarily switch markPaid to requireRole('super\_admin','hr\_manager','payroll\_admin').

**S (0.5 day)****next****Backend / Payroll squad**

**BIZ-008****High · P1****Risk 15****suspected****Duplicate final settlement / EOSB: no uniqueness or status guard on settlement creation****Module:** settlement · **Role:** super\_admin, hr\_manager · **L3 · T3 B5 U1**

settlement.create inserts a new final\_settlements row on every call with no check for an existing settlement for the same employee, and the table has no unique constraint or 'status/paid' field. offboarding.initiate also inserts an independent final\_settlements row. The same employee can therefore have multiple EOSB records, each a candidate for payout, with no idempotency or 'already settled' guard.

**Expected:** An employee should have at most one active final settlement; a repeat attempt should update the existing record or be rejected, and a settled/paid record must be immutable.

**Actual:** Every create/initiate call inserts a fresh row; nothing prevents duplicates.

**► Evidence (3)**

**Recommendation:** Add a unique/active constraint on employeeid (or a settlement status), check for an existing settlement before insert, and make a paid settlement immutable. Consolidate offboarding.initiate to reuse/upsert the same row.

**Workaround:** HR procedure to check for an existing settlement before creating one.

M (1 day incl. migration)

next

Backend / Payroll squad

**PRIV-003****High · P1****Risk 15****confirmed****Real production Supabase DB connection string (with password) stored in cleartext file in repo working tree****Module:** repo root · **Role:** N/A · **L3 · T5 B5 U5**

\_db\_url.txt at the repo root contains a live Supabase pooler connection string including the database password and the tenant search\_path (rukn\_energy\_services). The file is listed in .gitignore (so it is not committed to git history that was checked), but it sits in the developer working tree in plaintext, is trivially shared via backups/zips/screen-shares, and grants direct SQL access to production PII (iqama, IBAN, salary). A credential of this sensitivity must not live in a loose text file.

**Expected:** Production DB credentials live only in a secret manager / Vercel env vars, never in a working-tree file.

**Actual:** Full production connection string with password is readable from \_db\_url.txt.

**► Evidence (1)**

**Recommendation:** Rotate the Supabase database password immediately, delete \_db\_url.txt from disk, and confirm it never entered git history (git log --all -- \_db\_url.txt). Store the URL only in the secret manager.

**Workaround:** Delete the file now; rotate the credential regardless.

S

current

Security/DevOps

**PRIV-005****High · P1****Risk 15****confirmed****Password-reset token and user email written to server logs (account-takeover via log access)****corroborated: AUTH-005****Module:** apps/web (auth API) · **Role:** unauthenticated · **L3 · T4 B4 U5**

The request-reset handler logs the full reset URL (which embeds the raw single-use reset token) together with the user's email via console.info. Server logs are typically retained and broadly readable (Vercel log drains, observability tools). Anyone with log access can read the token within its 1-hour TTL and complete an account takeover; the email is also PII exposed in logs. The DB correctly stores only the token hash, so the log is the weak link.

**Expected:** Reset links must be delivered only to the user's mailbox and never logged; at most log a non-sensitive event id.

**Actual:** console.info("[auth] password reset link generated for", email, resetUrl) logs email + raw token.

**► Evidence (2)**

**Recommendation:** Wire the email delivery (TODO(email)) and remove the console.info entirely, or gate it behind NODE\_ENV !== 'production' with the token redacted. Never log tokens or email.

**Workaround:** Immediately restrict log access and set short log retention until fixed.

**S****current****Backend/Auth**

**QA-002****High · P1****Risk 12****confirmed****attendance.getSubtree fetches every employee in the tenant on every call instead of the requested org subtree****Module:** Attendance / GPS guide map · **Role:** department\_manager, hr\_manager, super\_admin · **L3 · T4 B3 U3**

The comment above the query says "Recursive CTE to get all employees under rootEmployeeId" but the actual Drizzle call has no `where` clause and no `limit`, so it loads the entire employees table (with department join) for the tenant on every request, then filters/builds the org tree in application memory (apps/web/trpc/routers/attendance.ts:722-758). A second unbounded findMany over attendanceRecords (with: {}) then follows for every employee in that full result set.

**Expected:** Only employees within the requested rootEmployeeId's subtree (and their latest attendance record) should be fetched from the database, ideally via a recursive CTE or a scoped WHERE clause.

**Actual:** ctx.db.query.employees.findMany({ with: { department: true } }) has no where/limit — it is a full tenant-wide employee scan on every guide-map request.

**► Evidence (2)**

**Recommendation:** Replace the full-table findMany with a recursive CTE (raw SQL via Drizzle's sql` helper) or an iterative BFS that only expands children of rootEmployeeId, and add a limit as a safety cap.

**Workaround:** None; acceptable today only because demo/pilot tenants are small.

**M****Next**

backend / performance-engineer

**QA-007****High · P1****Risk 12****confirmed**

## Zero automated tests for the two largest business-logic routers: retention.ts (1611 lines) and recruitment.ts (812 lines)

**Module:** Retention / Recruitment · **Role:** hr\_manager, department\_manager, super\_admin · **L4 · T3 B3 U3**

The vitest run (152 tests passed across 26 suite files) contains no test file for apps/web/trpc/routers/retention.ts or apps/web/trpc/routers/recruitment.ts. These are the two single largest source files in the repo (1611 and 812 lines respectively) and implement RBAC-scoped CRUD + workflow logic for goals/OKRs, performance reviews, skills/succession, engagement surveys, rewards/compensation, and the full recruitment pipeline (candidates through onboarding). No \_\_tests\_\_ directory reference to `retention` or `recruitment` domain logic exists anywhere in packages/ or apps/web/src/\_\_tests\_\_.

**Expected:** Business-logic routers of this size, especially ones gating access by requireCapability/requireRole (payroll:view\_company, performance:view\_team, etc.), should have integration tests covering at least the RBAC gate and core CRUD/workflow paths.

**Actual:** No test files reference these routers; coverage is 0% at the unit/integration level for both.

### ► Evidence (3)

**Recommendation:** Add integration tests per sub-router (goal, reviewCycle, skill, succession, engagement, reward, compensation for retention; job/candidate/application/interview/offer/onboarding/referral for recruitment) covering: RBAC gate rejection for unauthorized roles, pagination correctness, and at least one full create->read->update happy path per entity.

**Workaround:** None. [L](#) [Next](#) [tdd-guide / backend](#)

**QA-004****High · P1****Risk 10****confirmed**

## **pnpm audit: drizzle-orm pinned below the patched version for a high-severity SQL-injection advisory**

**Module:** Dependencies · **Role:** all · **L2 · T5 B4 U3**

`pnpm audit --audit-level=high` reports one high-severity advisory: "Drizzle ORM has SQL injection via improperly escaped SQL identifiers" (GHSA-gpj5-g38j-94v9), affecting drizzle-orm <0.45.2. packages/db/package.json pins "drizzle-orm": "^0.38.0" and the resolved version is 0.38.4 (12 dependency paths). `pnpm audit` (no level filter) additionally reports 4 moderate advisories: esbuild <=0.24.2 (dev-server request/response exposure, via drizzle-kit's esbuild-kit chain), next-auth 5.0.0-beta.25 (email misdelivery, patched at beta.30), and postcss <8.5.10 (XSS via unescaped </style> in stringify output, pulled in via Next.js 16.2.10).

**Expected:** Dependencies with known high-severity advisories should be upgraded to patched versions promptly, especially the ORM used for every tenant-scoped query.

**Actual:** drizzle-orm is pinned to ^0.38.0 (resolves to 0.38.4), below the 0.45.2 patched version.

### ► Evidence (2)

**Recommendation:** Upgrade drizzle-orm to >=0.45.2 (check for breaking changes against drizzle-kit and schema code), upgrade next-auth to >=5.0.0-beta.30, and let drizzle-kit pull in a patched esbuild-kit/esbuild chain; postcss will resolve once Next.js patches its bundled dependency.

**Workaround:** None; not yet exploited but exposure is real.

**M (upgrade + regression test)****Next**

dependency-manager / backend

**UX-003**

Medium · P2

Risk 20

confirmed

## Form labels not programmatically associated with inputs (missing htmlFor/id) across core HR forms

**Module:** Employees / Leave / Expenses / Settings / Payroll · **Role:** HR Manager, Employee, Department Manager, Super Admin · **L5 · T3 B3 U4**

Several of the app's oldest/core forms render plain ``<label>`` text with no ``htmlFor``, paired with ``<Input>``/``<select>`` elements that have no ``id``. This breaks the programmatic label-control relationship: clicking the label text does not focus the field, and screen readers announce the field with no accessible name (or fall back to the placeholder only). By contrast, newer forms (login, signup, recruitment/\*, retention/\*, offboarding/\*, documents/upload) consistently pair ``htmlFor`` with matching ``id``s, showing the pattern is known and used elsewhere in the codebase — it just wasn't applied to these forms.

**Expected:** Every ``<label>`` should carry ``htmlFor`` matching the paired input's ``id`` (or wrap the input), so assistive technology announces a clear field name and label-click focuses the control — as already done correctly in `apps/web/app/(auth)/login/login-form.tsx`.

**Actual:** Labels and inputs are visually adjacent but have no `id/htmlFor` link in the forms listed in Evidence.

### ► Evidence (3)

**Recommendation:** Add matching ``id``/`htmlFor`` pairs to all fields in the listed forms (or introduce a small ``<FormField>`` wrapper component in `packages/ui` that auto-generates and wires ids, then migrate these pages to it) to prevent regression.

**Workaround:** Sighted mouse/touch users are unaffected; only assistive-technology users are impacted.

M

Next

Frontend

**AUTH-003**

Medium · P2

Risk 12

confirmed

**No absolute session lifetime and stale role/privileges in JWT (no revocation on role change or deactivation)****Module:** `packages/auth/src/index.ts` · **Role:** authenticated · L3 · T3 B4 U3

The JWT session uses a 30-minute rolling idle window (maxAge 30m, updateAge 5m) with NO absolute cap, so an active session can be extended indefinitely. The jwt callback only re-hydrates employeeId on later requests - it never re-reads the user's role or verifies the account still exists/is enabled. Consequently a role downgrade, a role removal, or an account deletion is not reflected until the idle window lapses (up to 30 min), and there is no server-side revocation because the strategy is stateless JWT. The users table has no 'disabled/status' column, so an operator cannot force-logout or lock out an active token at all.

**Expected:** Role/authorization changes and account deactivation should take effect on the next request (re-read role from DB in the jwt callback or check a token version / session revocation list), and sessions should have an absolute maximum lifetime.

**Actual:** Role is copied only at first sign-in; later requests keep the old role. No absolute expiry, no revocation, no account-disabled concept.

**► Evidence (2)**

**Recommendation:** Re-read role (and an is\_active/token\_version) from the DB in the jwt callback on each request, add an absolute session cap, and add an account-status column so operators can force revocation.

**Workaround:** Reduce maxAge; instruct admins that role changes take effect after the idle window.

M (0.5-1d)

next

Auth

**AUTH-004**

Medium · P2

Risk 12

confirmed

## Account lockout silently disabled by schema drift - lockout columns absent from Drizzle schema

**Module:** `packages/auth/src/index.ts`, `packages/db/src/schema/public/users.ts` · **Role:** anonymous · L3 · T4 B3 U3

The durable account-lockout code reads/writes `failed_login_attempts` and `locked_until` via raw SQL wrapped in try/catch that swallows all errors and 'disables lockout until migrated' when the columns are missing. Those columns are defined ONLY in migration 0008; they are NOT present in the Drizzle schema (`users.ts`). Because CLAUDE.md documents ``pnpm db:push`` (schema-push) as a workflow, a push from the drifted schema can DROP `failed_login_attempts/locked_until`, silently turning brute-force lockout off with no error surfaced. Prior memory notes already flag attendance schema drift in this repo.

**Expected:** Lockout columns should be part of the source-of-truth Drizzle schema so `db:push` preserves them, and a missing-column condition should be logged loudly, not swallowed.

**Actual:** Columns live only in a hand-written migration; the schema object omits them; failures are swallowed and lockout becomes a no-op.

### ► Evidence (3)

**Recommendation:** Add `failedLoginAttempts` and `lockedUntil` to the Drizzle users schema so they are managed and preserved; replace the silent catch with a one-time warning log/metric.

**Workaround:** Use `db:migrate` (not `db:push`) in all environments; verify columns exist post-deploy.

S (2-4h)

next

DB/Auth

**BIZ-003**

Medium · P2

Risk 12

confirmed

**No audit trail on any expense mutation (create / approve / markPaid)****Module:** expense · **Role:** all staff roles · L4 · T2 B4 U1

The audit helper writeAudit is used for payroll runs, reopens and settlements, but the expense router never calls it. Approvals and payouts of company money leave no who/when/old/new record, so the self-approval and over-broad-payout risks (BIZ-001/002) cannot be detected or investigated after the fact.

**Expected:** Every financial state change (approve, reject, pay) should write an audit\_logs entry with actor, role, old/new status and amount, per the project's 'Audit log for all sensitive operations' rule.

**Actual:** No writeAudit calls exist anywhere in expense.ts; the module is entirely un-audited.

**► Evidence (2)**

**Recommendation:** Add writeAudit to approve, reject and markPaid (and ideally create/cancel), recording actor, role, amount and status transition.

**Workaround:** None at the application layer; rely on DB-level change data capture if available.

S (0.5 day)

next

Backend squad

**DB-006**

Medium · P2

Risk 12

confirmed

**All timestamps use `timestamp` (without time zone) — Riyadh (UTC+3) correctness risk for punch/approval/payroll times****Module:** packages/db (all schema files) · **Role:** employee / hr\_manager / payroll\_admin · L4 · T3 B3 U4

The generated migration 0000 contains zero timestamptz columns; every createdAt/updatedAt and event timestamp (punch\_in\_at, punch\_out\_at, approved\_at, paid\_at, completed\_at, sent\_at, resolved\_at, etc.) is `timestamp` without time zone. Serverless writers run in UTC while the business operates in Asia/Riyadh (UTC+3). Naive timestamps store whatever wall clock the writer used with no offset, so a punch recorded in UTC and displayed as Riyadh local (or vice versa) can be off by 3 hours — material for attendance late/early minutes, overtime, and payroll period boundaries. (Pure `date` columns for hire/termination/leave/period\_month are acceptable.) Note inconsistency: hand-written migration 0002 uses timestamptz, so the codebase is mixed.

**Expected:** Instant-in-time columns should be timestamptz (stored as UTC, rendered in Asia/Riyadh), consistently across schema and migrations.

**Actual:** timestamp without time zone everywhere in the Drizzle schema; timestamptz only in one ad-hoc migration.

**► Evidence (3)**

**Recommendation:** Migrate instant columns to timestamptz and standardize on UTC storage + Asia/Riyadh presentation. If mass migration is too costly short-term, enforce a single writer timezone (UTC) app-wide and document it; long-term convert to timestamptz.

**Workaround:** Ensure every writer sets time in UTC and every reader converts to Asia/Riyadh explicitly.

M-L (schema migration + app audit)

Next

Backend / DB platform

**DB-007****Medium · P2****Risk 12****confirmed**

## Field-encryption key falls back to a public hardcoded dev key outside NODE\_ENV=production; no key rotation/versioning

**Module:** packages/db crypto.ts · **Role:** system · L3 · T4 B4 U3

deriveKeys() throws only when NODE\_ENV === 'production'. In any other environment (preview, staging, CI, or a misconfigured prod where NODE\_ENV isn't exactly 'production'), a missing FIELD\_ENCRYPTION\_KEY silently uses DEV\_FALLBACK\_KEY = 'hrms-dev-only-insecure-field-encryption-key', which is in source control. PII (iqama/passport/IBAN) written in such environments is effectively plaintext. Separately, the construction has no key-versioning/rotation: the marker is a static 'encv1:' and the key label is fixed 'v1'; rotating FIELD\_ENCRYPTION\_KEY changes every deterministic ciphertext and makes all previously stored values fail GCM auth on read (decryptField throws), with no dual-key/rewrap path.

**Expected:** Missing key should fail closed in every deployed environment (not just production). A key id should be embedded so keys can be rotated with a rewrap/dual-read window.

**Actual:** Silent dev-key fallback for all non-production NODE\_ENV; no rotation mechanism.

### ► Evidence (2)

**Recommendation:** Fail closed whenever the app is deployed (e.g. require the key unless an explicit local/test flag is set), never fall back to a committed key in any hosted environment. Add a key id to the marker (encv1:<keyid>:) and implement dual-read (old+new key) + background rewrap for rotation. Validate presence at startup via packages/config env.

**Workaround:** Set FIELD\_ENCRYPTION\_KEY in every deployed environment including preview/staging. **M** (fail-closed is S; rotation support is M) **Next** **Security / Backend**

**DB-008**

Medium · P2

Risk 12

confirmed

**Single flat Drizzle migration mixes public and all tenant tables into one unqualified schema; no per-tenant migration runner****Module:** `packages/db drizzle + drizzle.config.ts` · **Role:** system · L3 · T4 B3 U2

`drizzle.config.ts` scans `schema/**/*.ts` (both public and tenant). The generated `0000_thin_freak.sql` contains 78 CREATE TABLE statements — public tables (accounts, sessions, users, tenants, verification\_tokens, invite\_token\_index) AND all tenant tables (employees, payslips, etc.) — with no schema qualifier and 106 ADD CONSTRAINT FKs, all in a single schema. `drizzle-kit migrate/push` targets the connection's default schema (public), so running it would create every tenant table in public. There is no per-tenant migration runner; ongoing schema evolution for tenants is done via ad-hoc DO-block or hardcoded migrations (DB-002), which is the root cause of the drift findings above.

**Expected:** A clear separation: public-schema migrations for shared tables, and a dedicated per-tenant migration/DDL pipeline replaying an authoritative bundle into each tenant\_<hex> schema. `drizzle-kit generate` should not blend the two.

**Actual:** One flat migration blends public + tenant tables; no mechanism keeps existing tenant schemas in sync with the Drizzle schema.

**► Evidence (3)**

**Recommendation:** Split schema generation: keep `drizzle` migrations for public-schema tables only, and build a first-class per-tenant migration runner (ordered SQL bundle applied with `search_path` per tenant, tracked in a per-tenant migrations table). Regenerate provisioning DDL from the tenant schema. Remove tenant tables from the public-targeted migration path.

**Workaround:** Never run `db:migrate/db:push` against the shared DB without a schema target; manage tenant DDL via scripts.

L (design + build per-tenant migration runner)

Next

Backend / DB platform

**DB-011**

Medium · P2

Risk 12

confirmed

**users.mfa\_secret (TOTP seed) is stored plaintext, not wrapped in encryptedText****Module:** packages/db public/users · **Role:** system / all users · L3 ·T4 B3 U3

The users table stores mfaSecret as plain text. Unlike employee iqama/passport/IBAN, the MFA shared secret is not wrapped in the encryptedText codec, so the TOTP seed sits at rest in cleartext. Anyone able to read the row can reconstruct the user's one-time codes.

**Expected:** MFA/TOTP secrets should be encrypted at rest (or stored in a dedicated secret store), consistent with the SEC-008 encryption applied to other PII.

**Actual:** mfa\_secret is plaintext text.

## ► Evidence (2)

**Recommendation:** Wrap mfa\_secret with the encryptedText codec (or a KMS-backed secret store). Since MFA secrets need no equality/uniqueness, prefer a randomized-IV variant rather than the deterministic codec to avoid equality leakage. Backfill existing rows.

**Workaround:** Restrict DB read access; rotate MFA enrollments if exposure suspected.

S-M

Next

Security / Auth

**PRIV-011**

Medium · P2

Risk 12

confirmed

**Audit log omits authentication, role-assignment, export, and PII-view events****Module:** apps/web (audit coverage) · **Role:** all · L4 ·T3 B3 U2

writeAudit is invoked only for employee create/update/delete, payroll runs, settlement, and PDPL self-export. Security- and privacy-relevant events are NOT persisted to audit\_logs: successful/failed logins and lockouts (only ephemeral console.warn), role assignment/changes via invites, MFA enable/disable, document/letter generation (PRIV-010), and PII/salary views. PDPL accountability and standard security monitoring expect durable auth and access-to-PII trails.

**Expected:** Login/logout, lockout, role changes, MFA changes, config changes, and PII/export access are written to a durable audit trail with actor/action/target/timestamp.

**Actual:** Auth events are only console-logged; role/MFA/export/view events are not audited.

## ► Evidence (3)

**Recommendation:** Emit audit\_logs entries for auth (login/logout/lockout/MFA), role/permission changes, config changes, and PII exports/views. Keep the existing tenant-scoped, read-only audit read surface.

**Workaround:** Ship auth events to a durable log store as an interim measure.

M

next

Backend/Auth

**UX-004**

Medium · P2

Risk 12

confirmed

**Required-field markers are visual-only and validation errors are not announced to assistive technology****Module:** Employees / Leave · **Role:** HR Manager, Employee · L4 · T2 B3 U3

Required fields are marked with a plain "\*" in the label text, but the underlying `<Input>/<select>` has no `required` or `aria-required` attribute. When client-side validation fails, the error is rendered into a plain `<div>/<p>` with no `role="alert"` or `aria-live` region, unlike the login form's error banner which correctly uses `role="alert"`.

**Expected:** Required inputs should carry `required/aria-required="true"`; the error summary should be in a `role="alert"` (or `aria-live="assertive"`) container so screen-reader users are told immediately when submission fails, matching `apps/web/app/(auth)/login/login-form.tsx:78-85`.

**Actual:** Submitting either form with missing/invalid data updates a plain div with the error text; a screen-reader user who has already moved focus away from the form is not notified anything happened.

**► Evidence (2)**

**Recommendation:** Add `role="alert"` to error containers and `required/aria-required` to fields already validated as mandatory in `handleSubmit`.

**Workaround:** Sighted users see the red banner directly below the button they just clicked.

S

Next

Frontend

**QA-006**

Medium · P2

Risk 10

confirmed

## **pnpm lint fails with 372 errors (almost entirely no-explicit-any / no-non-null-assertion) confined to the web app**

**Module:** apps/web (lint) · **Role:** n/a · L5 · T2 B2 U1

`pnpm lint` fails (turbo reports @hrms-app/web#lint exited 1). All 16 other packages (auth, config, date, db, demo, documents, email, leave, llm, payroll, qiwa, queue, ui, validators, etc.) lint clean. Only apps/web fails, with 387 problems (372 errors, 15 warnings) across ~84 files. The dominant rule is @typescript-eslint/no-explicit-any (the large majority of the 372 errors — 295 raw `any`/`as any` occurrences were found across apps/web by static grep), followed by @typescript-eslint/no-non-null-assertion (~30+ occurrences, e.g. attendance/me/page.tsx, payroll pages, leave.ts, invite.ts) and a smaller number of no-unused-vars.

**Expected:** `pnpm lint` should exit 0 across all packages, per the project's own ESLint config which forbids `any` and non-null assertions.

**Actual:** @hrms-app/web#lint exits with code 1; 372 errors reported.

### ► Evidence (3)

**Recommendation:** Triage by rule: replace `any` in router callback signatures (e.g. attendance.ts:60-61, department.ts:11, leave.ts:82) with the inferred Drizzle relation types; replace non-null assertions with explicit null checks or narrow the type upstream. Given the volume, tackle incrementally per-router rather than one large PR.

**Workaround:** None; lint is currently red.

L (incremental, per-router)

Backlog

frontend / build-error-resolver

**API-005**

Medium · P2

Risk 9

confirmed

## **File upload storage path traversal via unsanitized category field**

**Module:** REST API - upload · **Role:** authenticated · L3 · T3 B3 U3

In apps/web/app/api/upload/route.ts the object-storage key is built as `\${tenantId}/\${category}/\${timestamp}\_\${safeFileName}`. The file name is sanitized (safeFileName), but the client-supplied `category` string is used raw. A value such as `../<otherTenantId>` would resolve the stored object outside the caller's tenant prefix within the Supabase bucket, breaking tenant isolation of stored files.

**Expected:** The storage path segment derived from category must be validated against an allowlist (or sanitized the same way the filename is) so it cannot contain path separators or traversal sequences.

**Actual:** category is concatenated into the storage path without sanitization; `../` or slash characters allow escaping the tenant folder.

### ► Evidence (1)

**Recommendation:** Validate category against a fixed enum (e.g. policies, contracts, ids) or apply the same [^a-zA-Z0-9.\_-] sanitization used for the filename; reject null/empty category.

**Workaround:** Enforce an allowlist of category values server-side.

S

next

Backend/API team

**AUTH-006**

Medium · P2

Risk 9

confirmed

**MFA is opt-in only and not enforced by any policy (privileged roles can operate without MFA)****Module:** `packages/auth/src/index.ts, apps/web/trpc/routers/mfa.ts` · **Role:** all · L3 · T3 B3 U3

TOTP MFA is only enforced when the individual user has already enrolled a secret (if (user.mfaSecret)). There is no organization policy requiring MFA for high-privilege roles (super\_admin, hr\_manager, payroll\_admin), and nothing in signup/onboarding drives enrollment. A compromised password alone is sufficient for full admin access on any account that has not voluntarily enrolled.

**Expected:** Privileged roles should be required to enrol MFA (block or step-up until enrolled), configurable per tenant.

**Actual:** MFA gate is skipped entirely for any account without mfaSecret; no role-based enforcement exists.

## ► Evidence (1)

**Recommendation:** Add a tenant policy that requires MFA for privileged roles; block access (or force enrollment) until a secret is set.

**Workaround:** Operationally require admins to enrol MFA.

M (1-2d)

next

Auth/Product

**BIZ-004**

Medium · P2

Risk 9

suspected

**Leave balance can be over-drawn: balance is checked at submission but not re-validated at approval****Module:** `leave` · **Role:** employee (submitter) + approver · L3 · T2 B3 U2

The available-balance check runs only in create, against the balance at submission time. Two (or more) non-overlapping requests can each pass because the balance is not decremented until approval. At approval, `adjustLeaveBalance` decrements with `Math.max(0, ...)`, which silently floors a negative balance to 0 instead of rejecting, so the employee is granted more paid leave than their entitlement.

**Expected:** Approval should re-validate the current balance and refuse (or partially grant) when the request exceeds the remaining balance; balance must never be allowed to go negative silently.

**Actual:** No balance re-check at approval; decrement clamps to 0, hiding the over-draw.

## ► Evidence (3)

**Recommendation:** Move/duplicate the balance check into `updateStatus` at the approval boundary (reject if `days > current balance`) and remove the silent `Math.max` floor (or log when a floor would occur).

**Workaround:** HR manually reconciles balances before approving stacked requests.

S (0.5-1 day)

next

Backend / Leave squad

**BIZ-005**

Medium · P2

Risk 9

confirmed

**Leave approvals have no segregation of duties: an approver can approve their own leave****Module:** `Leave` · **Role:** `hr_manager`, `super_admin`, `department_manager` · L3 · T1 B3 U2

`updateStatus` authorises `super_admin`, `hr_manager` and `department_manager` to approve leave. There is no check that the approver is not the leave's own employee. A `department_manager` who heads their own department (dept scope check passes because their own department is in the managed set) can approve their own leave request; `hr_manager/super_admin` can approve their own outright.

**Expected:** The approver of a leave request must not be the requesting employee; self-approval should be blocked and routed to a higher authority.

**Actual:** Only role and (for `department_manager`) department scope are checked; `actor != subject` is not.

## ► Evidence (2)

**Recommendation:** In `updateStatus`, load the request's `employeeId` and reject when it equals the caller's `employeeId` (for the approval transition); require escalation to a higher approver.

**Workaround:** Policy control + monitor `approvedByUserId` vs `employee`.

S (0.5 day)

next

`Backend / Leave squad`**PRIV-008**

Medium · P2

Risk 9

confirmed

**No data-subject rectification / erasure / consent-withdrawal endpoints (PDPL rights incomplete)****Module:** `apps/web` (tRPC routers) · **Role:** `employee` · L3 · T2 B4 U3

A PDPL data-subject access (export) right is implemented (`employee.exportMyData`, audited). However there is no self-service or documented workflow for the right to correction, the right to destruction/erasure, or withdrawal of consent. Employee deletion is soft-delete only (`status=terminated`) which by design retains all PII indefinitely, and no anonymisation/retention-expiry job exists. Saudi PDPL grants data subjects rights to access, correction, and destruction; only access is covered in code.

**Expected:** Provide (or document a controlled process for) correction, erasure/anonymisation, consent withdrawal, and a defined retention/disposal schedule.

**Actual:** Only `exportMyData` exists; soft-delete retains PII with no erasure/anonymisation path.

## ► Evidence (2)

**Recommendation:** Add correction and erasure/anonymisation workflows (audited), a consent-withdrawal path, and a documented retention & disposal policy. Legal review required to define lawful retention periods (payroll/tax records vs erasable PII).

**Workaround:** Document a manual, audited DSR handling process in the interim.

L

backlog

`Product/Legal/Backend`

**PRIV-010**

Medium · P2

Risk 9

confirmed

### Salary certificate / experience letter generation exposes decrypted iqama + full salary and is not audited

**Module:** apps/web (tRPC document router) · **Role:** super\_admin/hr\_manager/hr\_specialist · L3 · T3 B3 U3

document.generateLetter builds a salary certificate / experience letter containing the employee's decrypted national ID (iqama) and full basic/housing/transport salary. This is a sensitive PII/financial export, but no writeAudit call is made, so there is no record of who generated a document containing another person's iqama and salary or when. writeAudit is used for employee/payroll/settlement mutations but not for this read/export.

**Expected:** Every generation/export of a document containing another person's PII/salary is audited (actor, subject, type, timestamp).

**Actual:** generateLetter returns HTML with iqama + salary and writes no audit entry.

#### ► Evidence (3)

**Recommendation:** Add writeAudit('document.generate', entityType 'employee', subject id, {type}) to generateLetter; consider the same for any PII-bearing read/export.

**Workaround:** N/A [S](#) [next](#) [Backend](#)

**QA-003**

Medium · P2

Risk 9

confirmed

### leave.runAccrual issues one sequential DB write per employee×leaveType combination instead of a bulk operation

**Module:** Leave / Accrual cron · **Role:** super\_admin, hr\_manager · L3 · T3 B2 U1

The `runAccrual` mutation computes accrual results for every active employee × leave type in memory, then loops over the results issuing one `await ctx.db.insert(...)` or `await ctx.db.update(...)` per row sequentially inside a for-loop, rather than batching the writes.

**Expected:** Bulk insert/update (e.g. a single multi-row insert with onConflict upsert, or Promise.all with a bounded concurrency) so the accrual run scales sub-linearly with employee count.

**Actual:** for (const result of results) { await ctx.db.insert(...) } / await ctx.db.update(...) — strictly sequential, one round trip per employee per leave type.

#### ► Evidence (1)

**Recommendation:** Batch inserts with a single multi-row INSERT ... ON CONFLICT DO UPDATE, or chunk the loop with Promise.all in small batches (e.g. 20 at a time) to reduce round trips while bounding connection usage.

**Workaround:** Fine at current demo/pilot scale. [S](#) [Backlog](#) [backend](#)

**QA-005**

Medium · P2

Risk 9

confirmed

**No timeout or retry on any external HTTP call (Qiwa gov API, Gemini, Claude)****Module:** Integrations / LLM · **Role:** all · L3 · T3 B2 U3

Every external `fetch()` call in the Qiwa client (`packages/qiwa/src/index.ts`, 8 call sites: auth token, employees, contracts, contract status, health check) and both LLM providers (`packages/llm/src/providers/gemini.ts:65`, `packages/llm/src/providers/claude.ts:56`) is a bare `await fetch(...)` with no `AbortController`/signal timeout and no retry/backoff logic.

**Expected:** External calls to third-party/government APIs and LLM providers should have a bounded timeout (e.g. 10-30s via `AbortController`) and at least one retry for transient network errors, so a slow upstream cannot hang a user-facing request indefinitely.

**Actual:** `fetch()` calls have no `signal`/timeout option anywhere in the `qiwa` or `llm` packages; a hung upstream connection will hang the calling `tRPC` procedure until the platform's own request timeout (if any) kicks in.

## ► Evidence (3)

**Recommendation:** Add an `AbortController`-based timeout (e.g. 15s for LLM, 20s for Qiwa) to every external fetch, and a single retry with backoff for network-level failures (not for 4xx application errors).

**Workaround:** The AI chat path already has a try/catch fallback to a stub reply for context-fetching, which limits blast radius for that one call; the main completion call itself has no fallback. S Next

backend / sre-engineer

**QA-009**

Medium · P2

Risk 9

confirmed

**Several tRPC routers with real business logic have no dedicated unit/integration tests (attendance, expense, compliance, department, settlement, notification)****Module:** Attendance / Expense / Compliance / Department / Settlement / Notification ·**Role:** all · L3 · T3 B2 U2

apps/web/trpc/routers/attendance.ts (788 lines: shifts, shift assignments, punches, exceptions, monthly reports, GPS subtree) has no matching unit/integration test file; it is only touched indirectly by two Playwright UI specs (attendance-guide-map.spec.ts, attendance-reports.spec.ts), which test rendering, not the underlying query/business logic (overtime calc, late/absence classification, subtree building). expense.ts, compliance.ts, department.ts, settlement.ts, and notification.ts routers similarly have no dedicated test file.

**Expected:** Routers implementing calculations (overtime, late minutes) or RBAC-scoped visibility rules (expense.ts's SEC-004 remediation for pending-approval visibility) should have logic-level tests independent of the UI.

**Actual:** Only e2e/UI-level coverage exists for attendance; no coverage at all for expense/compliance/department/settlement/notification routers.

**► Evidence (2)**

**Recommendation:** Prioritize a test for expense.ts's role-scoped visibility (given it references a prior IDOR fix) and attendance.ts's overtime/late calculation before adding new features to these routers.

**Workaround:** None. [M](#) [Backlog](#) [tdd-guide](#)

**Incomplete SEC-006 fix: attendance company reads and leave company reads remain companyProcedure (recruiter over-exposure)****Module:** tRPC routers attendance, leave · **Role:** recruiter · L3 · T3 B2 U3

The Phase 2 SEC-006 remediation applied requireCapability to recruitment, retention, settlement, document and ai reads, but did NOT touch the attendance and leave company-wide reads. attendance.list/monthlyReport/exceptions/assignment.list and leave.request.list/balance.list are still companyProcedure, which authorizes any non-employee/candidate staff role. The capability model grants attendance:view\_company and leave:view\_company to super\_admin/hr\_manager/hr\_specialist/payroll\_admin/department\_manager but NOT recruiter. A recruiter therefore reads all-employee attendance (attendance.list embeds each employee row and last workLocation) and all-employee leave, outside their capability set. The 03-permission-matrix.md itself flagged these as 'Over-permissive (recruiter)' — they were never closed.

**Expected:**

attendance.list/monthlyReport/exceptions/assignment.list should be requireCapability('attendance:view\_company') and leave.request.list/balance.list requireCapability('leave:view\_company'), consistent with the rest of Phase 2.

**Actual:** recruiter (no

attendance/leave company capability) can read every employee's attendance and leave records.

**► Evidence (3)**

**Recommendation:** Swap companyProcedure -> requireCapability('attendance:view\_company') on the four attendance reads and requireCapability('leave:view\_company') on leave.request.list/balance.list (mirror the getSubtree fix already applied).

**Workaround:** None client-side.[S](#)[Next sprint](#)[Backend](#)

**UX-005**

Medium · P2

Risk 9

confirmed

**Focus outline removed with no replacement on Super Admin tenant filter input****Module:** Super Admin · **Role:** Super Admin · L3 · T2 B2 U3

Across the codebase, every other instance of `focus:outline-none` is paired with a replacement indicator (`focus:ring-2`, `focus:border-\*`, etc. — 28 of 29 occurrences checked). The tenant filter input on the Super Admin page is the sole exception: it removes the native outline and supplies no visible focus state at all.

**Expected:** Keyboard users tabbing to the filter input should see a visible focus indicator (ring/border/box-shadow), consistent with every other input in the app.

**Actual:** Tabbing into the field shows no visible change; the input is indistinguishable from its unfocused state.

## ► Evidence (1)

**Recommendation:** Add `focus:ring-2 focus:ring-emerald-600` (or equivalent) matching the sibling filter/search inputs elsewhere in the app.

**Workaround:** Mouse users unaffected.

S

Next

Frontend

**UX-007**

Medium · P2

Risk 9

confirmed

**Hand-rolled "Save Map View" modal lacks dialog semantics, focus trap and Escape-to-close****Module:** Attendance / Guide Map · **Role:** HR Manager, Department Manager · L3 · T3 B2 U3

The app already ships an accessible modal primitive (`packages/ui/src/ui/dialog.tsx`, built on `@radix-ui/react-dialog`, with built-in focus trap, Escape handling and `aria-modal`) and uses it correctly elsewhere (`apps/web/app/(dashboard)/attendance/portal/page.tsx:249-328`). The Guide Map page's "Save This View" flow instead renders a plain `fixed inset-0` overlay `

` with no `role="dialog"`, no `aria-modal="true"`, no focus trap, and no keydown handler for Escape.

**Expected:** Opening 'Save This View' should trap focus inside the dialog, be announced as a dialog to screen readers, and close on Escape — matching the Punch Dialog pattern on `/attendance/portal`.

**Actual:** Tab can move focus out of the overlay into the page behind it; Escape does nothing; no dialog role is exposed.

## ► Evidence (2)

**Recommendation:** Replace the plain overlay with `Dialog`/`DialogContent` from `@hrms-app/ui`, consistent with `attendance/portal/page.tsx`.

**Workaround:** Mouse/touch users can click Cancel or click outside conceptually (though click-outside isn't wired either, since it's not a real dialog).

S

Next

Frontend

**UX-008**

Medium · P2

Risk 9

confirmed

### In-app help chat has no aria-live region for new messages

**Module:** Floating Help Chatbot · **Role:** All authenticated roles · L3 · T2 B2 U3

The floating Taazur Help chat appends new assistant/user messages to a scrolling `

` with no `aria-live` attribute. A screen-reader user who submits a question and then reviews other parts of the page will not be notified when the AI's reply arrives.

**Expected:** The message list (or a visually-hidden companion region) should be `aria-live="polite"` so new assistant replies are announced automatically, similar to how `apps/web/app/(dashboard)/loading.tsx` correctly uses `aria-live="polite"` for its skeleton state.

**Actual:** New messages (including the 'Thinking...' pending state) are silently inserted with no announcement.

► Evidence (1)

**Recommendation:** Add `aria-live="polite"` to the message container (line 177).

**Workaround:** Users can manually re-focus/re-read the panel after sending a message.

S

Next

Frontend

**UX-009**

Medium · P2

Risk 9

confirmed

### Destructive delete actions with zero confirmation step

**Module:** Attendance Guide Map / Recruitment Jobs · **Role:** HR Manager, Department Manager, Recruiter · L3 · T2 B3 U3

Two delete actions fire their mutation immediately on click with no confirmation of any kind — not even a native `confirm()`. This is inconsistent with the rest of the app, which guards comparable destructive actions (delete department, delete document, delete policy, revoke invite, cancel expense) behind a `window.confirm(...)` prompt.

**Expected:** Deleting a saved guide map or a job requisition should require an explicit confirmation step before the mutation fires, matching the pattern used for departments/documents/policies elsewhere in the app.

**Actual:** A single click on the trash icon (guide map) or the 'Delete' button (job requisition) immediately deletes the record.

► Evidence (3)

**Recommendation:** Add a confirmation step (ideally a themed `AlertDialog` rather than native `confirm` — see UX-010) before both mutations.

**Workaround:** None; deletion is immediate and irreversible from the UI.

S

Next

Frontend

**BIZ-009**

Medium · P2

Risk 8

confirmed

**Payroll updateStatus allows state-machine shortcuts and an un-audited completed->cancelled that re-opens the period lock****Module:** payroll · **Role:** super\_admin, hr\_manager · L2 · T3 B4 U1

updateStatus accepts any target status from updatePayrollRunSchema. It blocks completed -> (non-completed/non-cancelled) but still permits completed -> cancelled with no reason and no audit, and permits forward jumps such as draft -> completed that skip pre\_check/ready. Because the PAY-010 period lock exempts 'cancelled' runs, cancelling a completed run re-opens the month for a brand-new run, enabling a second payroll for the same period. Unlike the dedicated reopen mutation, updateStatus writes no audit record.

**Expected:** Status transitions should follow the defined lifecycle (draft -> pre\_check -> ready -> completed) with only the audited reopen able to unwind a completed run; any transition of a completed run should be audited.

**Actual:** Arbitrary forward jumps allowed; completed -> cancelled allowed silently with no audit; re-opens the period lock.

**► Evidence (2)**

**Recommendation:** Enforce explicit allowed transitions; route any completed-run change exclusively through the audited reopen (with reason); write an audit record for every status change; do not treat 'cancelled' as unlocking a period that already produced payslips.

**Workaround:** Operationally forbid using updateStatus on completed runs; use reopen only.

M (1 day)

next

Backend / Payroll squad

**BIZ-010**

Medium · P2

Risk 8

confirmed

### payslip.create allows manual injection of arbitrary payslip amounts, bypassing the calculation engine and audit

**Module:** payroll · **Role:** super\_admin, hr\_manager · L2 · T3 B4 U1

payslip.create accepts a client array of payslips (basic, housing, netPay, gosi, etc.) and inserts them directly with no recomputation against the GOSI/orchestrator engine, no verification that the run exists or is open, and no audit entry. Amounts are only bounded by the schema (basic/netPay positive, others  $\geq 0$ ), so arbitrarily large net-pay figures can be written straight into a run.

**Expected:** Payslip amounts should be produced by the orchestrator (as run.create does) or, for manual corrections, validated against the engine and audited, tied to a run in an editable state.

**Actual:** Raw client values are inserted verbatim; no engine cross-check, no audit.

#### ► Evidence (2)

**Recommendation:** Remove or lock down the raw create path; require corrections to go through an audited, engine-validated adjustment tied to a non-completed run.

**Workaround:** Disable payslip.create in the UI; only create payslips via run.create. S-M (0.5-1 day)

[next](#)[Backend / Payroll squad](#)**BIZ-011**

Medium · P2

Risk 8

suspected

### Compensation adjustment can be self-approved in a single call (client-supplied status + approvedById)

**Module:** retention (compensation) · **Role:** super\_admin, hr\_manager · L3 · T2 B4 U1

createCompensationAdjustmentSchema lets the caller set both status and approvedById directly, and the create mutation inserts them verbatim. An hr\_manager can create a salary adjustment already flagged status='approved' with approvedById = self in one request, with no separate approval workflow, no actor != approver check, and no audit entry.

**Expected:** A compensation adjustment should be created as 'proposed', approved by a distinct authorised approver, and the approval should be audited; approvedById should be set by the server from the approving session, not the client.

**Actual:** status and approvedById are accepted from the client and stored as-is; no approval step, no audit.

#### ► Evidence (2)

**Recommendation:** Force status='proposed' on create, ignore client approvedById, add a separate approve mutation requiring a different authorised approver, and audit the approval.

**Workaround:** Restrict comp-adjustment approval to super\_admin and review manually. M (1 day)

[next](#)[Backend / Retention squad](#)

**PRIV-001**

Medium · P2

Risk 8

confirmed

## CSP allows script-src 'unsafe-inline' (and style-src 'unsafe-inline'), defeating XSS containment

**Module:** apps/web (Next config / CSP) · **Role:** all · L2 · T4 B4 U4

The Content-Security-Policy sets script-src and style-src to include 'unsafe-inline'. 'unsafe-inline' on script-src means any injected inline `<script>` or event-handler attribute executes, so the CSP provides no meaningful defence-in-depth against XSS. For a PII app (iqama, IBAN, salary) an XSS payload could read the DOM/tRPC responses and exfiltrate. 'unsafe-eval' was correctly dropped and the code comment acknowledges a pending nonce rollout, but until nonces ship the primary XSS mitigation is absent.

**Expected:** script-src should use per-request nonces or hashes (no 'unsafe-inline'); style-src should move to nonces/hashes where feasible.

**Actual:** script-src 'self' 'unsafe-inline' and style-src 'self' 'unsafe-inline' are hard-coded.

### ► Evidence (2)

**Recommendation:** Adopt Next.js nonce-based CSP (middleware sets a per-request nonce, propagate to inline scripts) and remove 'unsafe-inline' from script-src. Migrate inline styles to nonces/hashes or external stylesheets.

**Workaround:** Keep object-src 'none', frame-ancestors 'none', base-uri 'self' (already present) and prioritise the nonce rollout the comment references.

M

next

Frontend/Platform

**API-007**

Medium · P2

Risk 6

confirmed

## In-memory rate limiter is per-instance and unbounded; CSRF skipped when Origin absent

corroborated: AUTH-007

**Module:** middleware · **Role:** anonymous · L3 · T2 B3 U2

middleware.ts implements rate limiting with a module-level Map (rateLimitMap). On serverless/edge (Vercel) each instance has its own Map, so limits are per-instance and largely ineffective under horizontal scaling; the Map also never evicts expired entries (unbounded memory growth). Additionally, the same-origin CSRF check only rejects when an Origin header is present and mismatched - requests with no Origin header (e.g. non-browser clients) bypass the check.

**Expected:** Rate limiting should use a shared store (Redis) so limits hold across instances, evict expired keys, and CSRF/same-origin protection should fail-closed for state-changing requests lacking a trustworthy Origin.

**Actual:** Limits reset per instance and per deploy; Map grows unbounded; missing Origin header skips the CSRF check.

### ► Evidence (2)

**Recommendation:** Back rate limiting with Redis (already a dependency via BullMQ) keyed by IP+route; evict expired keys; require a valid Origin/Referer (fail-closed) for state-changing custom routes, or rely on NextAuth CSRF tokens.

**Workaround:** Add an edge/WAF rate limit; require Origin header on state-changing routes.

M

next

Platform/Infra team

**UX-011**

Low · P3

Risk 12

confirmed

## No "skip to main content" link before the sidebar navigation

**Module:** Dashboard Shell · **Role:** All authenticated roles · L4 · T1 B2 U3

There is no skip link anywhere in the codebase (confirmed via repo-wide search). Every dashboard page places roughly 15-20 sidebar navigation links (Sidebar's ``, apps/web/components/sidebar.tsx:158-184) before the `` landmark in the DOM order, with no mechanism for keyboard/screen-reader users to jump directly to page content.

**Expected:** A visually-hidden-until-focused 'Skip to main content' link should be the first focusable element, targeting the `` element in dashboard-shell.tsx:60.

**Actual:** Keyboard users must tab through the entire sidebar nav (and header controls) on every single page load/navigation before reaching page content.

### ► Evidence (2)

**Recommendation:** Add a standard skip link as the first child of DashboardShell's root, pointing to an `id` on the `` element.

**Workaround:** None; mouse/touch users unaffected.

S

Next

Frontend

**DB-009**

Low · P3

Risk 9

confirmed

**leave\_balances has no uniqueness on (employee\_id, leave\_type\_id, year) and no timestamps — duplicate balances / leave double-counting possible****Module:** packages/db leave\_balances · **Role:** hr\_manager / employee · L3 · T3 B3 U3

leave\_balances defines id, employeeid (FK), leaveTypeid (FK), balance numeric(5,1), year integer — with no unique constraint across (employee\_id, leave\_type\_id, year) and no created\_at/updated\_at. Nothing at the DB level prevents duplicate balance rows for the same key, and there is no audit of when a balance changed.

**Expected:** Exactly one balance row per employee per leave type per year, enforced by a unique constraint, with change timestamps.

**Actual:** Duplicates are permitted; no temporal audit columns.

## ► Evidence (2)

**Recommendation:** Add UNIQUE (employee\_id, leave\_type\_id, year) and created\_at/updated\_at columns; dedupe existing data before applying. Include in provisioning DDL (DB-001).

**Workaround:** Enforce single-row invariant in application upsert logic (ON CONFLICT once the unique index exists). [S](#) [Next](#) [Backend](#)

**DB-010**

Low · P3

Risk 9

confirmed

**PII '\_enc' columns are plaintext TEXT at the DB level — encryption is enforced only by the app-layer Drizzle codec****Module:** packages/db employees + crypto.ts · **Role:** system · L3 · T3 B3 U2

iqama\_number\_enc / passport\_number\_enc / bank\_iban\_enc are ordinary TEXT columns; confidentiality depends entirely on every writer going through the encryptedText customType. The codec even passes through non-marked (legacy plaintext) values on read, so a plaintext write is invisible on read and silently defeats encryption-at-rest. Seed scripts and any db.execute raw path are realistic bypasses.

**Expected:** PII-at-rest guarantee should not depend on developer discipline; ideally a DB-level guard (e.g., a CHECK/trigger requiring the encv1: marker) or centralized write path.

**Actual:** Plaintext can land in \_enc columns with no error and no read-time signal.

## ► Evidence (2)

**Recommendation:** Add a DB CHECK/trigger enforcing the encv1: marker on \_enc columns (allowing a time-boxed legacy window), or route all writes through a single repository. Add a lint/test that fails if raw writes target \_enc columns. Complete the encrypt-pii-backfill and then tighten the constraint.

**Workaround:** Code review to ensure only Drizzle codec writes these columns; run backfill. [S-M](#)

[Next](#)[Security / Backend](#)

**UX-010**

Low · P3

Risk 8

confirmed

### Destructive-action confirmations rely on unstyled native `window.confirm()` instead of the app's own dialog

**Module:** Settings / Documents / Departments / Expenses · **Role:** HR Manager, Super Admin · L4 · T1 B2 U2

Six separate destructive flows use the browser-native, unstyled `confirm()` dialog rather than the app's own themed Dialog component (already available in packages/ui and used for the Punch Dialog on /attendance/portal). Native `confirm()` cannot be localized to match the user's chosen Arabic/English preference, doesn't match the Saudi-luxury visual language, and blocks the JS main thread.

**Expected:** Destructive confirmations should use the shared Dialog/AlertDialog pattern, with buttons/copy that respect `preferredLanguage`.

**Actual:** Confirmations show the browser's default OK/Cancel dialog with English-only, unstyled text regardless of the app's current language setting.

#### ► Evidence (3)

**Recommendation:** Introduce a shared `useConfirm()`/`ConfirmDialog` built on the existing Dialog primitive and migrate these six call sites to it.

**Workaround:** None needed — functionally works today.

M

Backlog

Frontend

**API-006**

Low · P3

Risk 6

confirmed

### Internal error strings leaked to clients (qiwa, health)

**Module:** REST API - qiwa, health · **Role:** anonymous · L3 · T2 B2 U1

Several REST handlers return raw error text to the caller. `/api/qiwa` returns `error.message` (which can include upstream Qiwa/DB detail) and `/api/health` returns `String(err)` on failure. In contrast the tRPC error formatter and the login/signup/reset routes correctly return generic messages. This leaks internal detail (hostnames, driver errors) to unauthenticated clients.

**Expected:** Client-facing errors should be generic; detailed errors logged server-side only (as done in `/api/auth/login`).

**Actual:** Raw `error.message` / `String(err)` is serialized into the HTTP response body.

#### ► Evidence (2)

**Recommendation:** Replace error bodies with a generic message and log the detail via `console.error` server-side.

**Workaround:** Wrap responses with a generic error string.

S

next

Backend/API team

**API-009**

Low · P3

Risk 6

suspected

**Retention update schemas allow client to set employeeId/status via input.data spread****Module:** tRPC - retention router / validators · **Role:** hr\_manager, department\_manager · L2 · T2 B3 U2

The retention router applies updates with `.set({ ...input.data, updatedAt })` across ~40 procedures. Several update schemas in packages/validators/src/retention.ts include mutable identity/state fields such as employeeId (optional) and status (optional). Because the whole validated object is spread into .set(), a caller can reassign a record to a different employeeId or force a status transition directly, bypassing the intended workflow. Department write-scoping helpers exist (scoping.ts assertManagesGoal/Review/...) and procedures are capability-gated, which limits blast radius, but the schemas should not expose employeeId/status as freely-settable update fields. Marked suspected pending confirmation that each update procedure calls the corresponding assert* helper.`

**Expected:** Update DTOs should allowlist only the fields a caller may change; immutable ownership keys (employeeId) and workflow state (status) should be excluded from generic update schemas or handled via dedicated, guarded transitions.

**Actual:** employeeId and status are present in update schemas and spread straight into .set().

**► Evidence (2)**

**Recommendation:** Remove employeeId (and status where it should be a guarded transition) from the \*Update schemas, or explicitly pick allowed fields before .set(). Ensure every update calls the matching assertManages\* scoping helper.

**Workaround:** Add explicit field picking in each retention update before writing.

M

next

Backend/API team

**AUTH-008**

Low · P3

Risk 6

confirmed

**MFA lacks recovery/backup codes and per-account TOTP attempt throttling**

**Module:** `apps/web/trpc/routers/mfa.ts`, `packages/auth/src/index.ts` · **Role:** authenticated · L2 · T2 B2 U3

MFA enrollment stores only a TOTP secret; there are no backup/recovery codes, so a user who loses their authenticator is locked out and requires manual DB intervention (clearing `mfa_secret`). Additionally, a wrong TOTP at login deliberately does not count toward account lockout (the password was correct), and the only throttle on TOTP guessing is the per-instance IP rate limit (see AUTH-007). With a +/-1 step window (3 valid codes per 1,000,000) online guessing is impractical but not rate-limited at the account level.

**Expected:** Provide single-use hashed backup codes at enrollment and throttle repeated failed TOTP attempts per account.

**Actual:** No backup codes; TOTP failures are unbounded at the account level.

## ► Evidence (2)

**Recommendation:** Generate 8-10 single-use backup codes (store hashed) at enrollment; add a per-account failed-TOTP counter that triggers a temporary MFA lock.

**Workaround:** Document an admin runbook to reset `mfa_secret` on request after identity verification.

M (0.5-1d)

backlog

Auth

**AUTH-009**

Low · P3

Risk 6

confirmed

**Self-serve signup mints a tenant super\_admin with no email verification or CR validation**

**Module:** `apps/web/app/api/auth/signup`, `apps/web/trpc/routers/auth.ts` · **Role:** anonymous · L3 · T2 B2 U2

Signup creates a tenant and inserts a user with role 'super\_admin' and no email verification (emailVerified is never set) and no CR-number validation. Anyone can register a tenant bound to an email they do not control. Blast radius is limited because super\_admin is tenant-scoped and cross-tenant registry access is separately gated by the PLATFORM\_ADMIN\_EMAILS allowlist (good), so the new super\_admin only controls its own freshly created (empty) tenant - but it does enable spam-tenant creation and unverified-email accounts. There is also no meaningful anti-automation on `/api/auth/signup` (only the loose 10/sec auth-route limit).

**Expected:** Require email verification before activating the account, validate the CR number format, and rate-limit signups.

**Actual:** super\_admin created immediately, unverified email, no CR validation.

## ► Evidence (1)

**Recommendation:** Add email verification before first login, validate CR number, and rate-limit signup (shared store).

**Workaround:** Monitor tenant creation; disable open signup if not required.

M (1-2d)

backlog

Auth/Product

**BIZ-006**

Low · P3

Risk 6

confirmed

**Leave requests can be back-dated: no lower bound on start date****Module:** Leave · **Role:** employee / HR on-behalf · L2 · T1 B3 U1

createLeaveRequestSchema only enforces endDate >= startDate; there is no minimum date. A request can be dated entirely in the past, decrementing the prior period's balance (the decrement year is derived from startDate), which can conflict with already-closed attendance/payroll periods.

**Expected:** Back-dated leave beyond a configurable grace window should be rejected or require explicit HR override, and should not silently affect a closed period.

**Actual:** Any past startDate is accepted; year is taken from startDate for the balance effect.

## ► Evidence (2)

**Recommendation:** Add a minimum-date rule (e.g. startDate >= today - N days) and block edits to closed periods; require HR override with audit for exceptions.

**Workaround:** HR review of any request with a past start date.

S (0.5 day)

backlog

Backend / Leave squad

**BIZ-014**

Low · P3

Risk 6

suspected

**Manager can author their own performance review/response (no actor != subject check)****Module:** retention (performance) · **Role:** department\_manager · L2 · T1 B2 U2

review.create and reviewResponse.create for a department\_manager are gated only by assertManagesEmployee / assertManagesReview (department scope). A manager whose own employee record is in a department they head can create or respond to a performance review about themselves, since the scope check passes. There is no check that the reviewer differs from the review subject.

**Expected:** A manager should not author or score their own performance review; self as subject should be rejected and escalated.

**Actual:** Only department-scope is enforced; actor != subject is not.

## ► Evidence (2)

**Recommendation:** Reject when input.employeeId (or the review's employeeId) equals the caller's employeeId; require a skip-level reviewer for a manager's own review.

**Workaround:** HR review of any self-authored review rows.

S (0.5 day)

backlog

Backend / Retention squad

**BIZ-015**

Low · P3

Risk 6

suspected

### Approval/payout/leave transitions lack an atomic status guard (race-condition double-processing)

**Module:** expense, leave, payroll · **Role:** approver roles · L2 · T2 B3 U1

State-changing updates read the current status, then update WHERE id = ... (not WHERE id AND status = expected). Two concurrent requests can both pass the status check and both apply the transition (e.g. approve+approve, or markPaid twice, or approve leave twice decrementing the balance twice). The guards are read-then-write rather than a conditional atomic update.

**Expected:** Transitions should use an atomic conditional update (WHERE id AND status = expected) or a transaction/row lock so only one concurrent actor can perform the transition.

**Actual:** Read-check-then-unconditional-update by id; no atomic guard.

#### ► Evidence (2)

**Recommendation:** Use conditional updates (.where(and(eq(id), eq(status, expected)))) and treat a zero-row result as 'already processed'; wrap balance-affecting leave approval in a transaction.

**Workaround:** Low real-world likelihood at current scale; prioritise after the higher-severity SoD/calc items. M (1 day across modules) backlog Backend squad

**PRIV-002**

Low · P3

Risk 6

confirmed

### Permissive CSP connect-src 'https:' and img-src 'https:' allow PII exfiltration to any HTTPS host

**Module:** apps/web (Next config / CSP) · **Role:** all · L2 · T3 B3 U3

connect-src permits 'https:' (any TLS origin) and img-src permits 'https:' (any TLS origin). Combined with script-src 'unsafe-inline' (PRIV-001), an injected script can beacon PII to an attacker-controlled https endpoint (fetch or an <img> pixel with data in the query string) without CSP interference.

**Expected:** connect-src and img-src should enumerate the specific origins the app actually needs (self + Supabase + Perplexity CDN + Sentry).

**Actual:** connect-src 'self' https: and img-src 'self' blob: data: https: are wide open to all HTTPS hosts.

#### ► Evidence (2)

**Recommendation:** Replace 'https:' with an explicit allow-list of required origins for connect-src and img-src.

**Workaround:** N/A S next Platform

**PRIV-006**

Low · P3

Risk 6

confirmed

**User email placed in reset URL query string (leaks via history / referrer)****Module:** apps/web (auth API) · **Role:** unauthenticated · L2 · T2 B2 U3

The reset link carries the user's email as a query parameter. Query strings land in browser history, proxy logs, and (for any off-site navigation) the Referer header. Email is PII under PDPL and need not be in the URL — the token alone identifies the request server-side.

**Expected:** Identify the reset by token only; resolve the email server-side from the token record.

**Actual:** email is included as a query parameter alongside the token.

## ► Evidence (1)

**Recommendation:** Drop the email param; look it up from the verificationTokens row by token hash during reset.

**Workaround:** N/A S next Backend/Auth

**PRIV-007**

Low · P3

Risk 6

confirmed

**Authentication failures log user email (PII) via console, bypassing the redacting logger****Module:** packages/auth · **Role:** unauthenticated · L3 · T2 B2 U2

Failed-login and MFA-failure paths log the account email through console.warn. A redacting pino logger exists in packages/config (redacts authorization/cookie/password) but is not used here, and its redact list does not cover email or PII fields anyway. Emails are PII and accumulate in logs on every failed attempt.

**Expected:** Log a hashed/opaque account identifier, not the raw email; route through the redacting logger.

**Actual:** console.warn('[auth] failed login attempt for', email) and MFA-failure log the email.

## ► Evidence (3)

**Recommendation:** Remove email from these log lines (or hash it) and route auth logging through the redacting logger; extend redact paths to cover PII.

**Workaround:** N/A S next Backend/Auth

**UX-006**

Low · P3

Risk 6

confirmed

**Icon-only "clear search" button has no accessible name****Module:** Super Admin · **Role:** Super Admin · L3 · T1 B1 U2

The small 'X' button that clears the tenant filter text renders only a lucide ``<X>`` icon with no ``aria-label``, so it has no accessible name.

**Expected:** The button should expose an accessible name, e.g. ``aria-label="Clear filter"``.**Actual:** Screen readers announce it as an unlabeled 'button'.

## ► Evidence (1)

**Recommendation:** Add ``aria-label="Clear filter"`` to the button.**Workaround:** Sighted users can infer purpose from the icon.

S

Backlog

Frontend

**UX-012**

Low · P3

Risk 6

confirmed

**Shared Table component's header cells omit explicit `scope="col"`****Module:** packages/ui Table · **Role:** All roles viewing tabular data · L3 · T1 B1 U2

The shared ``TableHead`` renders a bare ``<th>`` with no ``scope`` attribute. Most browsers/screen readers infer ``scope="col"`` for ``<th>`` inside a ``<thead>``, so this is not a hard failure, but explicit scope is recommended best practice, particularly for the app's data-dense HR tables which are otherwise the most tabular-heavy content in the product.

**Expected:** ``TableHead`` should default to ``scope="col"`` (with an override prop for any future row-header use).**Actual:** No scope attribute is emitted.

## ► Evidence (1)

**Recommendation:** Default ``scope="col"`` in `TableHead`, consumed once by all ~15+ tables in the app.**Workaround:** None needed; most AT infers this correctly already.

S

Backlog

Frontend / Design system

**API-010**

Low · P3

Risk 4

suspected

**AI chat forwards client-supplied model string to the LLM provider****Module:** tRPC - ai.chat.send · **Role:** authenticated · L2 · T2 B2 U1

ai.chat.send passes input.model (validated only as z.string().min(1).optional()) straight to getLlmClient().complete({ model: input.model }). If packages/llm does not allowlist the model, a client could request an arbitrary or more expensive model, driving up cost. The message array is well-bounded (max 20 messages, 8000 chars each) and temperature is capped, which limits other abuse.

**Expected:** The model override should be constrained to a server-side allowlist of approved models.

**Actual:** Any non-empty model string is forwarded to the provider.

## ► Evidence (2)

**Recommendation:** Validate model against z.enum([...approvedModels]) or resolve it server-side per topic; ignore client model unless it is in the allowlist.

**Workaround:** Drop input.model server-side and pick the model per provider.

S

backlog

AI/Backend team

**API-011**

Low · P3

Risk 4

confirmed

**company/profile PATCH and company/invite POST parse body without Zod****Module:** REST API - company · **Role:** company admin · L2 · T1 B2 U1

Both handlers destructure request.json() manually instead of validating with a Zod schema. company/profile PATCH writes industry/companySize/website with no length/format bounds (unbounded strings). company/invite POST validates email only via `email?.trim()` (no email-format check) though it does allowlist the role. Both are auth+role gated so the risk is limited to authenticated tenant admins, but this deviates from the project's Zod-everywhere convention and permits unbounded/malformed data.

**Expected:** All request bodies validated with a Zod schema (bounded strings, emailSchema).

**Actual:** Manual destructuring; no schema; unbounded strings; email format unchecked on invite.

## ► Evidence (2)

**Recommendation:** Introduce Zod schemas (bounded strings + emailSchema) and safeParse the body, matching signup/reset routes.

**Workaround:** Add length caps and an email regex check inline.

S

backlog

Backend/API team

**API-012**

Low · P3

Risk 4

confirmed

**Password-reset link (raw token) logged to server console in all environments****Module:** REST API - request-reset · **Role:** anonymous · L2 · T2 B2 U2

request-reset generates a single-use reset token and, because email delivery is still a TODO, logs the full reset URL (including the raw token) via console.info for every request in every environment. Anyone with access to server/log aggregation can read valid reset tokens and take over accounts within the token TTL (1 hour).

**Expected:** Reset tokens/links must never be logged; deliver via the email package. Any diagnostic logging should be gated to non-production and must not include the raw token.

**Actual:** console.info logs email + full reset URL with the raw token unconditionally.

## ► Evidence (1)

**Recommendation:** Remove the token from logs; wire up @hrms-app/email delivery. If a dev fallback is needed, gate it to NODE\_ENV!=production and log only that a link was generated (no token).

**Workaround:** Delete the console.info line and rely on email delivery.

S

next

Auth team

**AUTH-010**

Low · P3

Risk 4

best-practice

**Reset token transmitted in URL query string (history / Referer / access-log leakage)****Module:** apps/web/app/api/auth/request-reset, reset-password UI · **Role:** anonymous · L2 · T2 B2 U2

The reset link carries the raw token (and email) as URL query parameters. Tokens in URLs are recorded in browser history, may leak via the Referer header to any third-party asset on the reset page, and land in server/proxy access logs. Token TTL (1h), single-use and hashed storage limit the window, but query-string delivery is discouraged.

**Expected:** Prefer delivering the token via a path segment with a strict no-referrer policy, or accept the token in a POST body / fragment; ensure Referrer-Policy: no-referrer on the reset page.

**Actual:** email + raw token are in the query string and read client-side from useSearchParams.

## ► Evidence (2)

**Recommendation:** Set Referrer-Policy: no-referrer on the reset route and keep the reset page free of third-party assets; consider a path-based token.

**Workaround:** Short TTL (already 1h) and single-use (already enforced) limit exposure.

S (1-2h)

backlog

Auth/Frontend

**AUTH-011**

Low · P3

Risk 4

confirmed

**Diagnostics endpoint discloses env wiring (AUTH\_SECRET length, DB host/port) outside production****Module:** apps/web/app/api/auth/debug · **Role:** anonymous · L2 · T2 B2 U1

The debug endpoint returns AUTH\_SECRET length, AUTH\_URL/NEXTAUTH\_URL values, DB hostname and port, NODE\_ENV and demo-mode state to anonymous callers. It is correctly disabled in production unless ENABLE\_DEBUG\_ENDPOINT=true (good), but in any non-production/preview environment it is fully open and leaks internal hostnames and the secret length. Preview deployments of a SaaS are frequently internet-reachable.

**Expected:** Diagnostics should require authentication (platform-operator) even in non-prod, and should never echo secret metadata such as AUTH\_SECRET length.

**Actual:** Anonymous GET returns env wiring in all non-prod environments.

## ► Evidence (1)

**Recommendation:** Require platform-operator auth for this endpoint in all environments and drop authSecretLength/dbHost/dbPort from the payload.

**Workaround:** Ensure preview deployments are access-controlled.

S (1-2h)

backlog

Platform

**BIZ-012**

Low · P3

Risk 4

confirmed

**Offer outcome transitions (send/accept/decline) set status unconditionally with no state-machine guard****Module:** recruitment · **Role:** super\_admin, hr\_manager · L2 · T2 B2 U2

The offer send/accept/decline mutations each set the status directly by id with no check of the current status. An offer can be moved to 'accepted' without ever being 'sent', re-accepted after being declined, or flipped between accepted and declined, and update can set any status with side-effect timestamps. There is no enforced draft -> sent -> accepted/declined lifecycle.

**Expected:** Only a 'sent' offer should be acceptable/declinable; terminal states (accepted/declined) should be immutable; illegal transitions rejected.

**Actual:** Status is overwritten unconditionally regardless of prior state.

## ► Evidence (2)

**Recommendation:** Guard each transition on the required prior status and make accepted/declined terminal; reject illegal transitions with a clear error.

**Workaround:** Operational discipline; review offer audit history.

S (0.5 day)

backlog

Backend / Recruitment squad

**BIZ-013**

Low · P3

Risk 4

suspected

**Referral creation trusts client-supplied referrerEmployeeId and rewardAmount (self-crediting)****Module:** recruitment · **Role:** any staff role (protectedProcedure) · L2 · T1 B2 U1

referral.create is a bare protectedProcedure and inserts input directly; createReferralSchema carries a required client-supplied referrerEmployeeId and an optional rewardAmount. A user can create a referral crediting any employee (including themselves) and propose the reward amount. The reward payout transition (status='reward\_paid') is HR-gated via referral.update, so money is not moved directly, but the referrer-of-record and proposed amount are attacker-controlled.

**Expected:** referrerEmployeeId should default to the caller's own employee id (or be validated), and rewardAmount should be set by policy/HR, not the submitter. **Actual:** Both fields are taken verbatim from the client at create.

**► Evidence (2)**

**Recommendation:** Default referrerEmployeeId to the session employee (mirror the recognition.create SEC-010 fix) and derive rewardAmount from the reward policy rather than the client.

**Workaround:** HR verifies referrer and amount before marking reward\_paid.

S (0.5 day)

backlog

Backend / Recruitment squad

**PRIV-012**

Low · P3

Risk 4

confirmed

**Notifications: no unsubscribe / channel preferences; payslip email exposes net pay****Module:** apps/web + packages/email · **Role:** employee · L2 · T2 B2 U2

The notifications table supports email/sms/in\_app channels but there is no user preference/unsubscribe surface — a data subject cannot opt out of a channel. The PayslipReady email template renders net pay ('{netPay} SAR') in the message body, so salary lands in the mail transport/inbox if/when dispatch is wired. Note: no email dispatch (resend.emails.send) is currently implemented anywhere, so templates are defined but unsent; this is both a functional gap and a reason to fix the payslip content before go-live.

**Expected:** Per-user notification preferences/opt-out; sensitive emails avoid embedding salary figures (link to the app instead).

**Actual:** No preferences model; payslip email body includes net pay; no dispatch implemented.

## ► Evidence (2)

**Recommendation:** Add a per-user notification-preferences table with opt-out and an unsubscribe path; remove the salary figure from the payslip email (link to the secured payslip instead) before wiring dispatch.

**Workaround:** Keep email dispatch disabled until content/preferences are addressed.

M

backlog

Product/Backend

**PRIV-013**

Low · P3

Risk 4

confirmed

**Audit log retains salary old/new values as plaintext JSON****Module:** packages/db + apps/web audit helper · **Role:** super\_admin/hr\_manager · L2 · T2 B2 U2

For salary changes, writeAudit records old/new salary values in the audit\_logs.newValue / oldValue jsonb columns in plaintext. This is inherent to an audit trail and access is restricted (HR Manager / super admin, tenant-scoped), but note the employee salary/PII fields are encrypted at rest in the employees table while their historical values sit unencrypted in audit\_logs. Any DB-level exposure of audit\_logs discloses salary history.

**Expected:** Sensitive audit payloads are protected commensurately with the source columns (access control at minimum; consider encryption/tokenisation for salary snapshots).

**Actual:** Salary before/after values stored as plaintext jsonb in audit\_logs.

## ► Evidence (2)

**Recommendation:** Restrict audit\_logs at the DB layer; consider encrypting or masking salary snapshots in audit payloads, or storing field-change markers instead of raw values where full fidelity is not required.

**Workaround:** Ensure audit\_logs access is tightly controlled and monitored.

M

backlog

Backend/Security

**QA-010**

Low · P3

Risk 4

confirmed

**Only 3 Playwright e2e specs exist; no end-to-end coverage of payroll run, leave approval, or recruitment pipeline****Module:** E2E · **Role:** all · L2 · T2 B2 U2

apps/web/e2e/ contains exactly 3 Playwright specs: auth.spec.ts, attendance-guide-map.spec.ts, attendance-reports.spec.ts (plus a non-test production-performance.mjs script). There is no e2e coverage for payroll run creation/approval, leave request/approval, employee onboarding, or the recruitment pipeline — all multi-step, high-value user flows.

**Expected:** Critical multi-step flows (per CLAUDE.md's own testing rule requiring E2E for critical flows) should have at least a smoke-level Playwright spec.

**Actual:** E2E coverage is limited to auth and two attendance screens.

**► Evidence (1)**

**Recommendation:** Add smoke-level Playwright specs for: leave request -> manager approval, payroll run creation -> approval -> payslip view, and recruitment candidate -> offer -> onboarding happy path.

**Workaround:** docs/e2e-status.md may already track this gap — cross-check before duplicating planning. [M](#) [Backlog](#) [e2e-runner](#)

**QA-011**

Low · P3

Risk 4

confirmed

**Two router files exceed the project's own 800-line file-size guideline****Module:** apps/web/trpc/routers · **Role:** n/a · L2 · T2 B1 U1

apps/web/trpc/routers/retention.ts is 1611 lines (2x the 800-line max) and recruitment.ts is 812 lines. Both are flat single-file routers covering many unrelated sub-domains (retention.ts alone spans goals, review cycles, reviews, skills, succession planning, engagement surveys, rewards, compensation, and talent reviews).

**Expected:** Per the project's coding-style guideline (200-400 lines typical, 800 max), large routers should be split by sub-domain into separate files/modules.

**Actual:** retention.ts and recruitment.ts remain single monolithic files well past the stated maximum.

**► Evidence (2)**

**Recommendation:** Split retention.ts into per-sub-domain router files (goal.ts, review.ts, skill.ts, succession.ts, engagement.ts, reward.ts, compensation.ts) merged via createTRPCRouter composition, mirroring the pattern already used for top-level routers in trpc/index.ts.

**Workaround:** None needed urgently; purely a maintainability concern. [M](#) [Backlog](#)

[refactor-cleaner](#)

**QA-012**

Low · P3

Risk 4

confirmed

**Leading-wildcard LIKE search on employees.fullName cannot use a standard index****Module:** Employees list · **Role:** all · L2 · T2 B1 U2

The employee search filter uses ``like(schema.tenant.employees.fullName, \`${input.search}%\`)` with a leading wildcard, which prevents a standard btree index from being used even if one existed (see QA-001) — Postgres would need a trigram (`pg_trgm`) or full-text index to serve this efficiently.

**Expected:** Name search should use an index-friendly strategy (trigram index, or a prefix-only search) once tenant employee counts grow large.

**Actual:** Leading-wildcard LIKE forces a full scan of the filtered result set for the search term.

**► Evidence (1)**

**Recommendation:** Add a `pg_trgm` GIN index on `employees.full_name` (`CREATE EXTENSION pg_trgm; CREATE INDEX ... USING gin (full_name gin_trgm_ops)`) if/when tenant sizes grow past a few thousand employees.

**Workaround:** Acceptable at current demo/pilot scale.

S

Backlog

database-optimizer

**RBAC-007**

Low · P3

Risk 4

confirmed

**recruitment.application.create and recruitment.referral.create trust full client payload on bare protectedProcedure (residual SEC-010)****Module:** tRPC router recruitment · **Role:** any staff role (payroll\_admin, department\_manager, hr\_specialist included) · L2 · T2 B2 U2

SEC-010's create-spoofing sub-item was only partially remediated. `offer.accept/decline`, `recognition.create/update` and `surveyResponse.create/update` were fixed, but `recruitment.application.create` and `recruitment.referral.create` remain bare `protectedProcedure` that insert the raw client payload (including `referrerEmployeeId`) with no session binding and no `recruitment:manage` gate. Any staff role that passes `canAccessProcedure` (recruiter, but also `payroll_admin`, `department_manager`, `hr_specialist`) can create applications/referrals with a spoofed referrer to credit an arbitrary employee. It is not reachable by employee/candidate (not in their allowlists), so impact is staff-internal.

**Expected:** Bind referrer/owner ids to the session where applicable and gate creation to `requireRole(super_admin, hr_manager, recruiter)` (`recruitment:manage`), consistent with the other recruitment mutations.

**Actual:** Staff roles without `recruitment:manage` can create applications/referrals with client-controlled owner fields.

**► Evidence (2)**

**Recommendation:** Change both to `requireRole(super_admin, hr_manager, recruiter)` and bind `referrerEmployeeId` to the session for self-referrals.

**Workaround:** None client-side.

S

Backlog

Backend

**RBAC-008**

Low · P3

Risk 4

confirmed

**ai.assistant.list / getByld remain companyProcedure (assistant config readable by all staff roles)**

**Module:** tRPC router ai · **Role:** any staff role (recruiter, department\_manager, payroll\_admin) · L2 · T2 B1 U1

Phase 2 moved ai.assistant.create/update/delete to requireCapability('settings:manage') but left the corresponding reads (ai.assistant.list, ai.assistant.getByld) on companyProcedure, so any staff role can read the AI assistant configuration (prompts, model settings). Low sensitivity, but inconsistent with the settings:manage gating applied to the writes.

**Expected:** Reads of settings-managed configuration should be requireCapability('settings:manage') (or at least align with who may manage them).

**Actual:** Any staff role reads AI assistant configuration.

**► Evidence (1)**

**Recommendation:** Gate ai.assistant.list/getByld on requireCapability('settings:manage') to match the write path.

**Workaround:** None client-side.

S

Backlog

Backend

**UX-013**

Low · P3

Risk 4

best-practice

**Table rows use `role="link"` plus a nested real `<button>`, an unconventional interactive pattern****Module:** Employees · **Role:** HR Manager, HR Specialist · L2 · T1 B1 U2

Each employee `<tr>` is given `role="link"`, `tabIndex={0}`, an `aria-label`, and both click and Enter/Space keydown handlers to navigate to the employee's profile — genuinely more accessible than a typical unlabeled `div-onClick` pattern, and this is the only such pattern found in a repo-wide scan for `onClick` divs (none found). However, `role="link"` on a `<tr>` that performs a client-side `router.push` (not a real href) is non-standard — links normally support right-click/open-in-new-tab and show a URL on hover/status bar, neither of which this provides. The row also contains a real nested `<Button>` ("View") that duplicates the row's own action, i.e. two overlapping interactive targets on one row.

**Expected:** Either wrap the row's primary cell content in a real `<a href="/employees/{id}">` (progressively enhanced with client routing) so link semantics are accurate, or use `role="button"` if it must stay a synthetic control — and drop the redundant nested 'View' button once the row itself is fully operable.

**Actual:** Row exposes `role="link"` without real link affordances, plus a redundant nested button performing the same navigation.

**► Evidence (2)**

**Recommendation:** Consider using `role="button"` (since it triggers a client-side action, not a navigable href) and drop the duplicate 'View' button, or convert the whole row to a real anchor for proper link semantics.

**Workaround:** Row is already keyboard operable via Enter/Space; not blocking.

S

Backlog

Frontend

**API-008**

Low · P3

Risk 3

confirmed

**Raw SQL string interpolation in demo seed endpoint****Module:** REST API - seed · **Role:** operator (token) · L1 · T3 B2 U1

apps/web/app/api/seed/demo-data/route.ts builds INSERT/UPDATE statements by string-interpolating values (emp.id, t.id, balance, year, department ids) directly into SQL passed to adminDb.execute(). The interpolated values are trusted (UUIDs/numbers read from the DB, not user input) and the endpoint is guarded by a NODE\_ENV!=production check plus MIGRATION\_TOKEN, so exploitability is low - but the pattern is an injection-prone anti-pattern and the only string-concatenated SQL in the codebase alongside the migrate routes.

**Expected:** All queries should use Drizzle's parameterized builder or sql`` with bound parameters, never raw string interpolation - per the project's 'never raw SQL' convention (CLAUDE.md).

**Actual:** Values are concatenated into raw SQL strings via .replace() and template literals.

## ► Evidence (1)

**Recommendation:** Delete the demo seed/migrate endpoints after the demo (as their own comments state), or rewrite using parameterized Drizzle inserts.

**Workaround:** Ensure NODE\_ENV=production in all live environments (currently the case) and remove the endpoints post-demo. S backlog Backend/API team

**API-013**

Low · P3

Risk 3

confirmed

**Unauthenticated /api/vitals accepts partially-validated payload (crash + log injection)****Module:** REST API - vitals · **Role:** anonymous · L2 · T1 B1 U1

/api/vitals is unauthenticated (acceptable for a beacon) and validates payload.name against an allowlist, but does not validate that payload.value is a number before calling payload.value.toFixed(2) - a non-numeric value throws outside the try/catch, yielding an unhandled 500. payload.path is logged unsanitized (log-injection vector). Low sensitivity, but input is not fully validated (no Zod).

**Expected:** Validate the whole payload with Zod (name enum, value number, bounded path) before use.

**Actual:** Only name is checked; value type and path content are trusted.

## ► Evidence (1)

**Recommendation:** Validate the payload with a Zod schema (value: z.number(), path bounded/stripped of control chars) and wrap logging in the try/catch.

**Workaround:** Coerce/guard value and strip control characters from path. S backlog

Frontend/Platform team

**PRIV-004**

Low · P3

Risk 2

confirmed

**Vercel OIDC JWT tokens stored in local .env.local / .env.production.local****Module:** repo root · **Role:** N/A · L1 · T2 B2 U1

VERCEL\_OIDC\_TOKEN (a signed JWT) is present in both .env.local and .env.production.local. These files are gitignored and the tokens are short-lived (exp claim ~12h from iat), so residual risk is low, but they are auto-generated bearer tokens sitting on disk.

**Expected:** Ephemeral OIDC tokens are fine when gitignored and short-lived; ensure they are never committed or shared.

**Actual:** Tokens present on disk (gitignored, not committed).

## ► Evidence (1)

**Recommendation:** No action beyond confirming .gitignore coverage (already present). Do not paste these files into shared channels.

**Workaround:** N/A S backlog DevOps

**AUTH-012**

Info · P4

Risk 4

best-practice

**Password minimum length of 8 and no breached-password / max-length-independent checks****Module:** packages/validators/src/auth.ts · **Role:** all · L2 · T2 B1 U1

The shared password policy requires 8+ chars with upper/lower/digit/special (good composition, and a 128 max that safely bounds bcrypt input). For a product holding payroll/PII, an 8-char minimum is on the low side; there is no check against known-breached passwords (e.g. HaveIBeenPwned k-anonymity) and no MFA-required compensating control for admins (see AUTH-006). Note bcrypt truncates input at 72 bytes - acceptable here given the 128-char cap and complexity, but worth documenting.

**Expected:** Raise the minimum to 12 for privileged roles and/or add a breached-password screen; document the bcrypt 72-byte behavior.

**Actual:** 8-char minimum, no breach check.

## ► Evidence (1)

**Recommendation:** Increase minimum length (12 for privileged roles), add a breached-password check, and pair with enforced MFA for admins.

**Workaround:** Enforce MFA for privileged roles (AUTH-006). S (2-4h) backlog Auth

**DB-012**

Info · P4

Risk 4

best-practice

**Deterministic (SIV) field encryption leaks plaintext equality — deliberate, documented tradeoff for the iqama unique index****Module:** packages/db crypto.ts · **Role:** system · L2 · T2 B1 U1

encryptField derives the AES-GCM IV as HMAC(ivKey, plaintext) truncated to 12 bytes (a synthetic-IV/SIV construction), making ciphertext deterministic. This is required so the iqama partial unique index and equality lookups work on the encrypted column. The known cost is that equal plaintexts produce equal ciphertexts (equality is observable). The code is a correct authenticated-encryption construction: AES-256-GCM, 16-byte tag, domain-separated enc/iv subkeys via HMAC labels, tamper rejection on read (verified by crypto.test.ts). Recording as a strength with a documented caveat, not a defect.

**Expected:** For a column needing a unique index, deterministic encryption is the accepted design; equality leakage is inherent and acknowledged in code comments.

**Actual:** As designed. Note: fields that do NOT need equality/uniqueness (e.g., bank\_iban\_enc, and mfa\_secret per DB-011) would be better served by randomized IVs to avoid unnecessary equality leakage.

## ► Evidence (2)

**Recommendation:** Keep deterministic encryption for iqama (unique-index requirement). For fields not needing equality (IBAN, MFA secret), consider a randomized-IV variant to remove equality leakage. Document the field-by-field choice.

**Workaround:** None required. S (optional) Backlog Security

**PRIV-009**

Info · P4

Risk 2

confirmed

**Consent capture labelled 'GDPR' in a Saudi PDPL product****Module:** apps/web (recruitment) · **Role:** recruiter/hr · L1 · T1 B2 U1

Candidate creation captures explicit consent (a genuine strength), but the field and validation message are named for GDPR while the product's governing regime is Saudi PDPL. The consent language/basis should reference PDPL to be accurate for the jurisdiction.

**Expected:** Consent wording and field naming reference the applicable PDPL basis.

**Actual:** Field named gdprConsent with a GDPR-worded validation message.

## ► Evidence (2)

**Recommendation:** Rename to reflect PDPL and have legal confirm the consent wording/basis for KSA.

**Workaround:** N/A S backlog Product/Legal

**RBAC-009**

Info · P4

Risk 2

confirmed

**Debug endpoint /api/auth/debug leaks env/config metadata (production-gated)****Module:** REST route (custom) · **Role:** anonymous (non-production only) · L1 · T2 B1 U1

GET /api/auth/debug returns AUTH\_SECRET length, AUTH\_URL/NEXTAUTH\_URL values, DB host/port, NODE\_ENV and DEMO\_MODE. It is correctly disabled in production (returns 404 unless ENABLE\_DEBUG\_ENDPOINT === 'true'), so risk is limited to non-production environments. Recorded for completeness; if a non-prod environment is internet-reachable this leaks the auth-secret length and DB host.

**Expected:** Diagnostic endpoints should not be shipped as public HTTP routes; if kept, restrict by network/auth even outside production.

**Actual:** In non-production, anonymous callers receive env/config metadata.

## ► Evidence (1)

**Recommendation:** Delete the route or require an operator token even in non-production; never expose AUTH\_SECRET length or DB host.

**Workaround:** Ensure NODE\_ENV=production and ENABLE\_DEBUG\_ENDPOINT unset in all internet-reachable deployments.

S

Backlog

Backend / Platform

## 9. Remediation roadmap

### Immediate — 0 to 48 hours (Critical / active exposure)

- Delete or authenticate GET /api/auth/check-tenants (gate behind PLATFORM\_ADMIN\_EMAILS; add to the middleware matcher; return 404 to anonymous). Confirmed live-exploitable. [RBAC-001]
- Delete or authenticate /api/qiwa — it proxies the Qiwa government API with server credentials and no auth. A gated tRPC equivalent already exists. [API-001]
- Rotate the exposed Supabase production database password, delete the on-disk \_db\_url.txt, and confirm it is absent from git history. [PRIV-003]
- Remove /api/auth/login (the app authenticates via NextAuth) or route it through the same logout + rate-limit + MFA controls. [AUTH-002]
- Stop logging password-reset tokens; wire email delivery. [PRIV-005]

### Urgent — 3 to 7 days (High)

- Fix tenant provisioning (generateTenantDDL) to create all ~24 tables plus the audit-log trigger, iqama unique index and performance indexes — or block open signup until fixed. New tenants are currently broken. [DB-001, DB-002, DB-003, DB-004]
- Patch drizzle-orm to  $\geq 0.45.2$  to clear the high-severity SQL-injection advisory. [QA-004]
- Add 'actor is not the subject' checks to expense/leave/compensation approvals and restrict expense.markPaid to HR/payroll. [BIZ-001, BIZ-002, BIZ-005, BIZ-011]
- Fix the compounding leave-accrual calculation and add a uniqueness/status guard to end-of-service settlements. [BIZ-007, BIZ-008]
- Stop returning plaintext salary and \_enc PII to recruiter/department\_manager, and department-scope employee.getById. [API-004]
- Add DB indexes to the employees table and bound attendance.getSubtree with a where/limit. [QA-001, QA-002]

### Short term — 1 to 4 weeks (Medium)

- Finish the capability pass on attendance/leave reads still using companyProcedure. [RBAC-004]
- Tighten CSP: remove script-src/style-src 'unsafe-inline' (use nonces/hashes). [PRIV-001]
- Move auth rate limiting to a shared store (Redis is not configured in prod) and enforce MFA for privileged roles. [API-007, AUTH-006]
- Add an absolute session cap and re-read role / support revocation on each request. [AUTH-003]
- Convert instant columns to timestamptz for Riyadh/UTC correctness. [DB-006]
- Un-skip the tenant-isolation regression tests and add tests for the retention and recruitment routers. [QA-007, QA-008]
- Sanitize the upload category path segment to prevent cross-tenant path traversal. [API-005]
- Fix RTL layout in the dashboard shell, wire the login Arabic toggle, and add label associations + delete confirmations. [UX-001, UX-002, UX-003, UX-009]

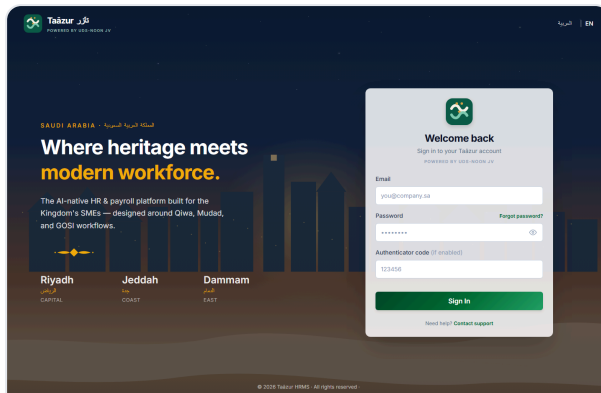
### Medium term — 1 to 3 months (hardening & compliance)

- Extend audit-log coverage to authentication, role changes, exports and PII views; provision the audit table for every tenant. [PRIV-011, DB-003]
- Implement PDPL data-subject rights (erasure, rectification, consent withdrawal) and a documented retention policy — legal review recommended. [PRIV-008]
- Add timeouts and retries to all external calls (Qiwa, Gemini, Claude, Resend). [QA-005]
- Reduce the ~295 explicit any usages driving the 372 lint errors; restore a green lint gate in CI. [QA-006]
- Add MFA backup codes, a breached-password check, and raise the minimum length for privileged roles. [AUTH-008, AUTH-012]
- Add an is\_active/status column and token versioning for real-time deprovisioning. [AUTH-003]

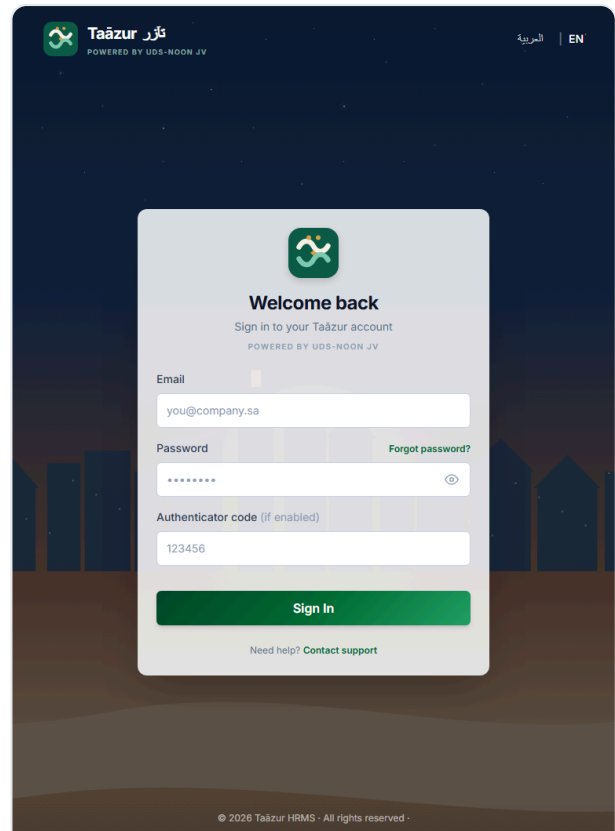
## 10. Screenshot evidence

45 screenshots were captured live from the running application across 4 roles and 4 viewports (all four demo logins succeeded). A representative subset is shown below; the complete set is in `evidence/screenshots/` with the index in `evidence/screenshot-index.json`.

### 01 Discovery (2 of 4)

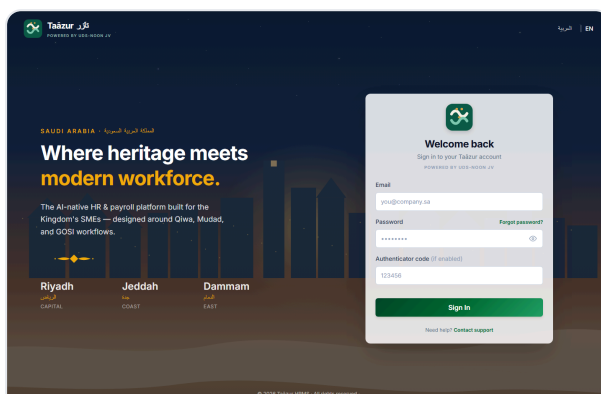


DISC-desktop-1440 · login-page-desktop-1440



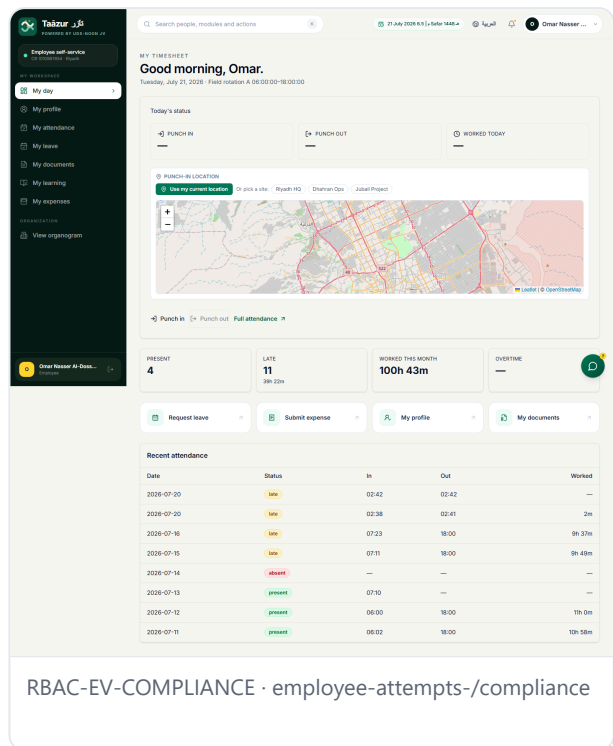
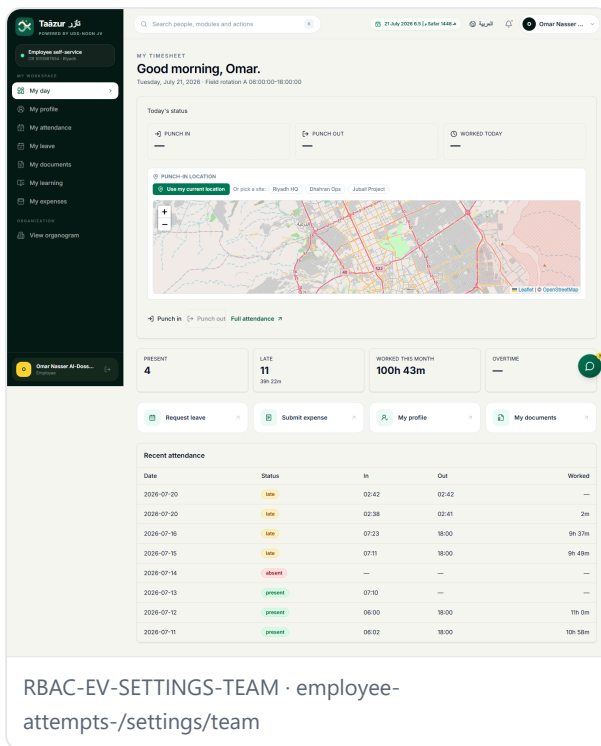
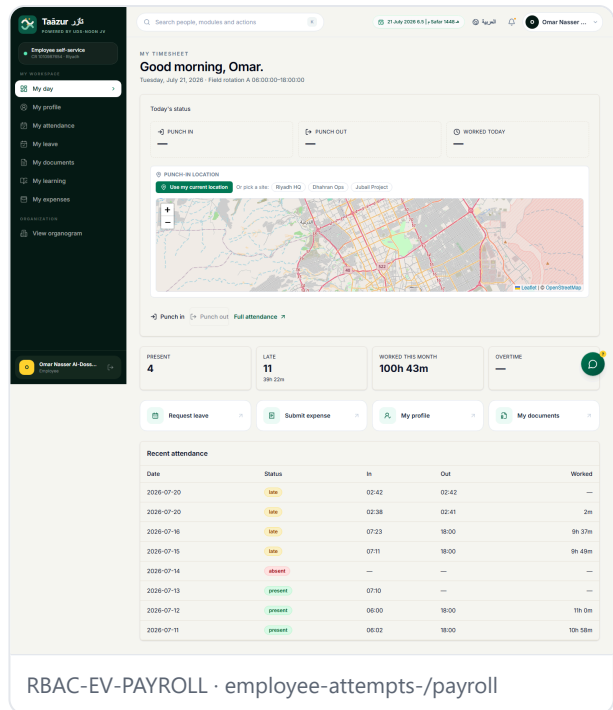
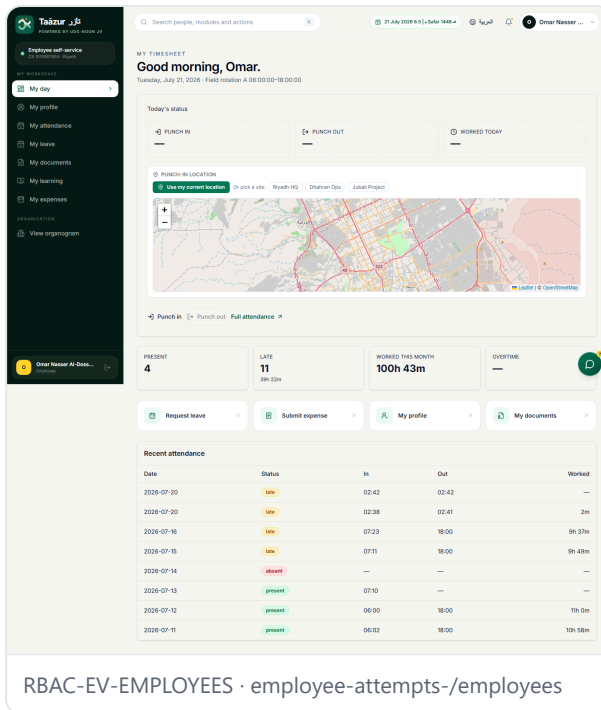
DISC-tablet-768 · login-page-tablet-768

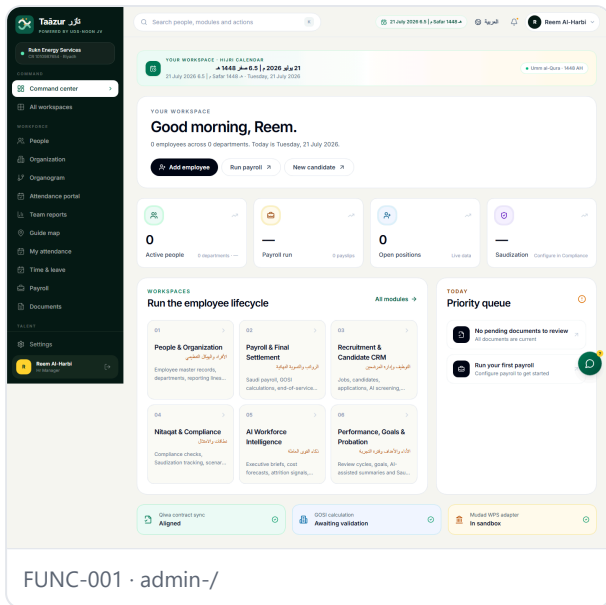
### 02 Authentication (1 of 1)



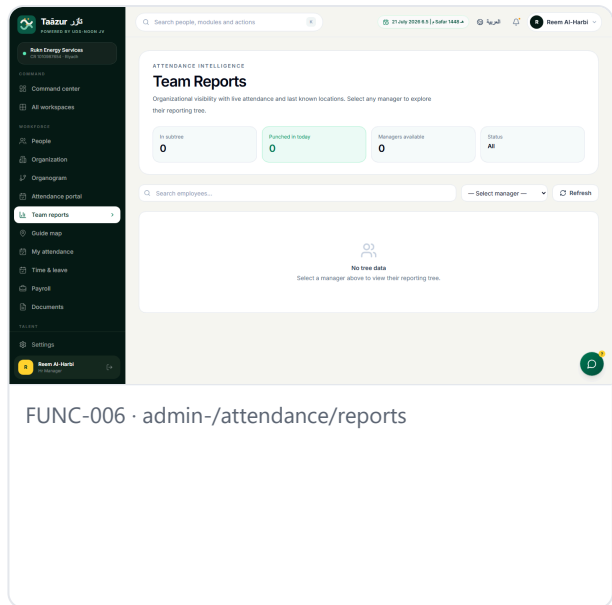
AUTH-EV-anon-redirect · anon-protected-route-redirect

### 03 RBAC (4 of 6)

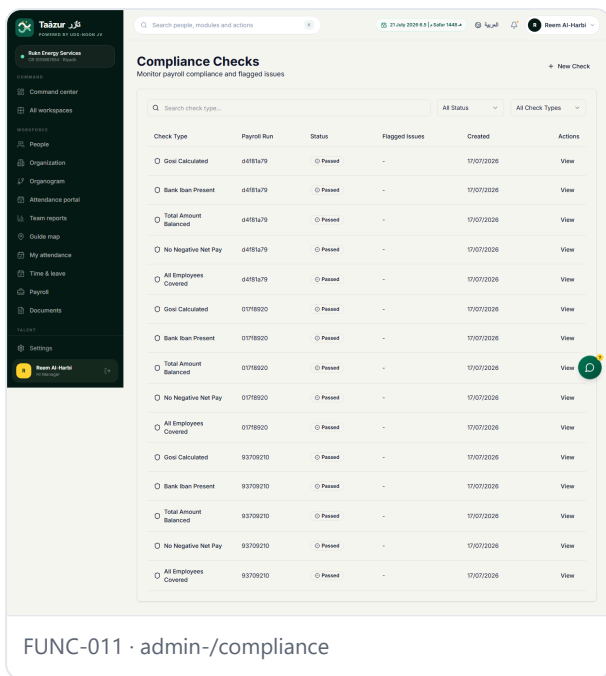




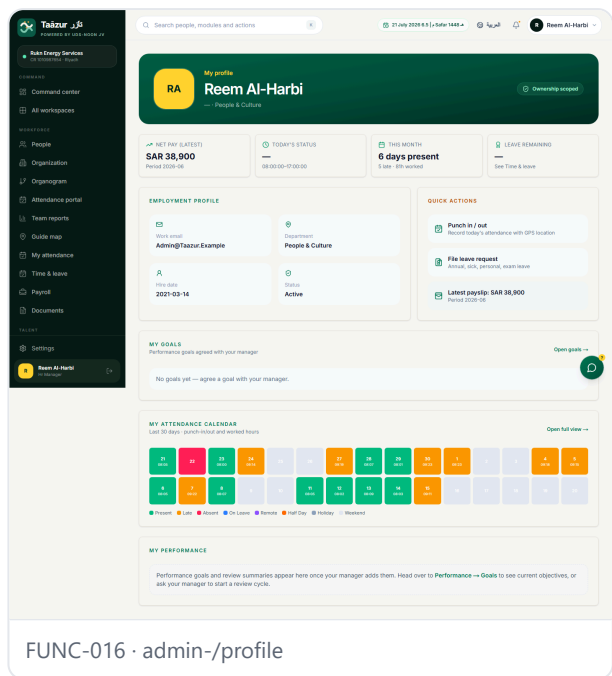
FUNC-001 · admin-/



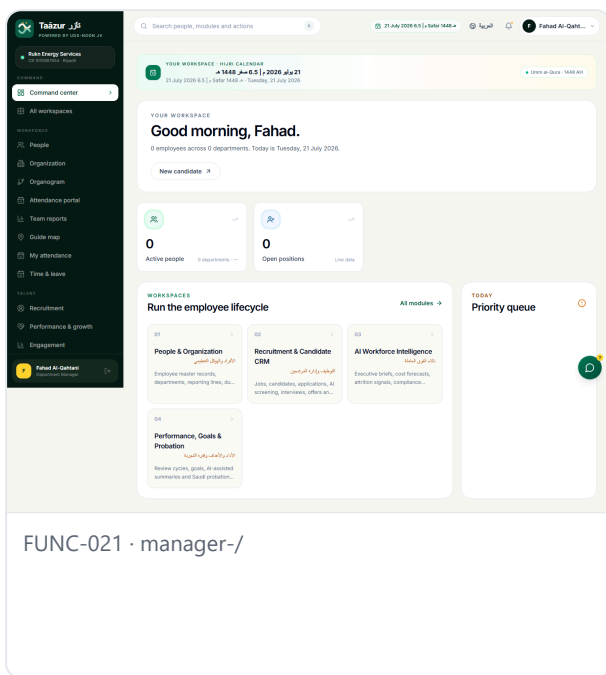
FUNC-006 · admin-/attendance/reports



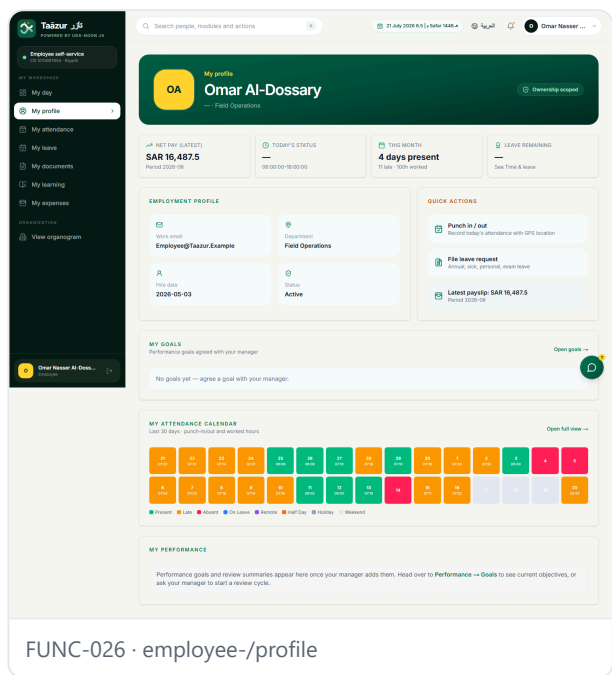
FUNC-011 · admin-/compliance



FUNC-016 · admin-/profile

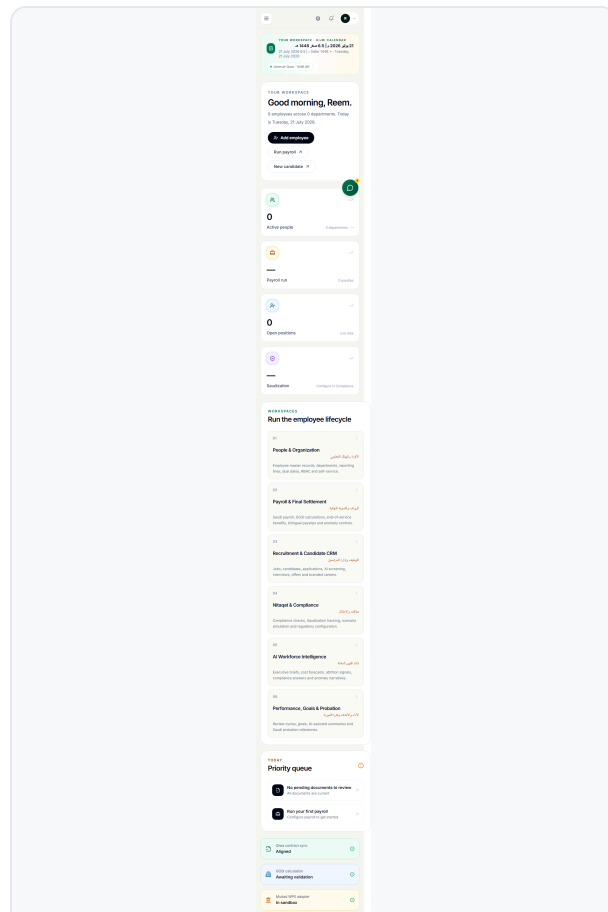


FUNC-021 · manager-/

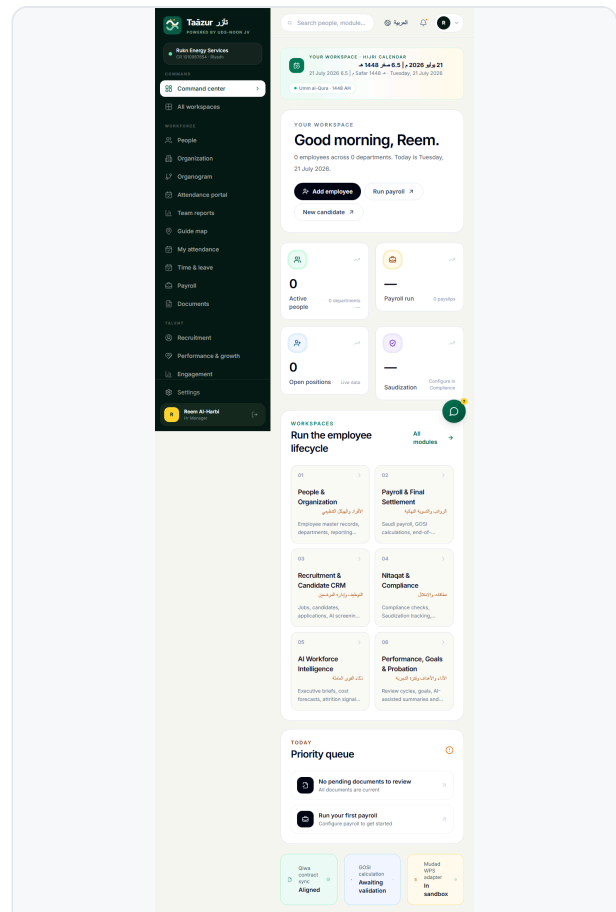


FUNC-026 · employee-/profile

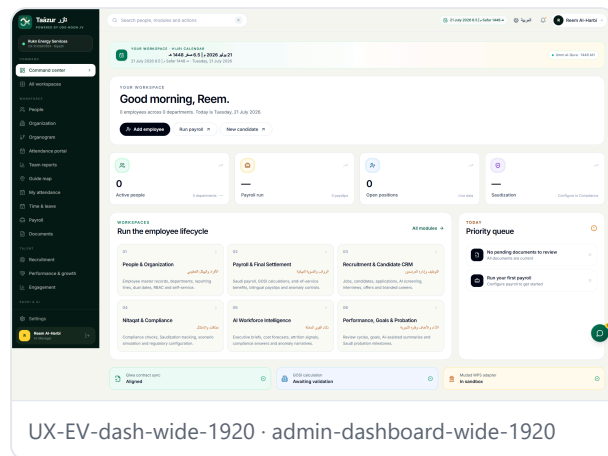
## 05 UI UX (3 of 3)



UX-EV-dash-mobile-375 · admin-dashboard-mobile-375



UX-EV-dash-tablet-768 · admin-dashboard-tablet-768



UX-EV-dash-wide-1920 · admin-dashboard-wide-1920

## 10 Errors (1 of 1)

404

**Page not found**

The page you're looking for doesn't exist or may have moved.

[Back to Home](#)

ERR-EV-404 · unknown-route-404

## 11. Strengths & conclusion

---

### What the app does well

- Server-side authorization is genuinely enforced on the tRPC API — an unauthenticated call returns 401 (verified live).
- Multi-tenant isolation is schema-per-tenant, resolved from the session tenantId and never from client input.
- PII (iqama, passport, IBAN) is encrypted at rest with AES-256-GCM behind a fail-closed production key, with a correct deterministic-SIV tradeoff so the iqama unique index still works.
- Complete modern security headers: HSTS preload, CSP, X-Frame-Options DENY, nosniff, object-src none, frame-ancestors none. Source maps are disabled (.map returns 403) and the debug endpoint 404s in production — all verified live.
- bcrypt cost 12, hashed single-use 1-hour reset tokens, no user enumeration on login, durable account lockout with backoff, and correct RFC 6238 TOTP.
- All monetary columns are numeric (no floats); payroll, GOSI and end-of-service are computed server-side and never trust the client; the payroll period lock has an audited reopen; statutory deduction caps are applied.
- Append-only audit-log trigger (where provisioned) and a role-gated, tenant-scoped, read-only audit API; notifications are strictly self-scoped; the invite route's role allowlist blocks super\_admin escalation (SEC-001 fixed).
- CSV formula-injection is neutralized on Mudad exports and uploads are magic-byte validated.
- Accessible Radix UI primitives, an exemplary login form, thoughtful empty states and focus-visible defaults; clean typecheck across all 17 packages and a deep payroll/statutory test suite.

Taazur HRMS is architecturally sound and, in its tRPC core, well-secured — the team clearly knows how to build secure multi-tenant software. The failures cluster in two avoidable places: a handful of custom REST routes that never received the same authorization treatment as the tRPC layer, and a tenant-provisioning path that drifted out of sync with the schema. Both are contained and fixable within days.

The headline score — 61/100, 'significant improvement required' — reflects real, currently-reachable exposure, chiefly the unauthenticated tenant-registry and Qiwa endpoints, rather than a weak foundation. After the Immediate and Urgent items in Section 9, a re-test should place the product in the low-to-mid 80s.

Recommendation: treat the four Critical items and the exposed database credential as a same-day fix, complete the Urgent items this week, and run a focused re-test of the REST surface and the tenant-provisioning path before onboarding any real customer tenant onto a non-demo deployment.

Generated 2026-07-21 · hrms-app holistic audit · 90 findings · white-box + live black-box.